

# Walnut: A Generic Framework with Enhanced Scalability for BFT Protocols

Lei Tian<sup>1</sup>, Chenke Wang<sup>1</sup>, Yu Long<sup>1</sup>, Xian Xu<sup>2</sup>, Mingchao Wan<sup>3</sup>,  
Chunmiao Li<sup>3</sup>, Shi-Feng Sun<sup>1</sup>, and Dawu Gu<sup>1</sup>

<sup>1</sup> School of Computer Science, Shanghai Jiao Tong University, Shanghai, China  
{leileitian, w\_chenke, longyu, shifeng.sun, dwgu}@sjtu.edu.cn

<sup>2</sup> Department of Computer Science and Engineering, ECUST, Shanghai, China  
xuxian@ecust.edu.cn

<sup>3</sup> Beijing Academy of Blockchain and Edge Computing, Beijing, China  
chainmaker@baec.org.cn, chunmiaoli1993@gmail.com

**Abstract.** The performance of traditional BFT protocols significantly decreases as  $n$  grows ( $n$  for the number of replicas), and thus, they support up to a few hundred replicas. Such scalability issues severely limit the application scenarios of BFT. Meanwhile, the committee sampling technique has the potential to scale the replica size significantly by selecting a small portion of replicas as the committee and then conveying the consensus results to the rest. However, this technique is rarely used in BFT, and there is still a lack of methods to scale the traditional BFT protocol being deployed to support more replicas rather than the costly re-deployment of new protocols. This paper introduces Walnut, a secure and generic committee-sampling-based modular consensus. Specifically, we use the verifiable random function for committee election and integrate committee rotation with the consensus. This resulting construction ensures that each selected committee is of a fixed size and acknowledged by all replicas, even in a partially synchronous network. For Walnut, we provide a rigorous definition and outline the necessary properties of each module to achieve safety and liveness. To clarify Walnut’s effectiveness, we apply this framework to HotStuff to obtain the Walnut-HS protocol, together with a proof of fit-in. We also implement Walnut-HS and compare its performance with HotStuff, using up to 100 Amazon EC2 instances in WAN. The experiments show that Walnut-HS can easily scale to 1,000 replicas with only a slight performance degradation, while HotStuff performs poorly or even breaks when  $n > 200$ . Besides, Walnut-HS performs well in comparison with Hotstuff for small-scale experiments. For example, the peak throughput of Walnut-HS is at most 38.6% higher than HotStuff for  $n = 100$ .

**Keywords:** BFT · Scalability · Committee sampling.

## 1 Introduction

As a long-studied protocol for reaching multi-party consistency, the Byzantine fault-tolerant (BFT) protocol can achieve distributed consensus in the permis-

sioned environment, even in the presence of malicious or faulty replicas (also called nodes) [24]. Among them, protocols working in the partially synchronous setting where messages can be delayed by an unknown bound (e.g., [5, 14]), have been extensively studied and widely used in real-life applications. PBFT [7] is the first practical Byzantine fault-tolerant protocol and achieves  $O(n^3)$  communication complexity (where  $n$  is the number of participating replicas). Subsequently, a series of BFT protocols have emerged to reduce the communication complexity. Particularly, HotStuff [36] achieves  $O(n)$  communication using aggregated signatures and further enhances the performance through a pipelined workflow. These works mostly adopt a timeout mechanism to handle the possible network fluctuations.

**Scalability issue in BFT protocols.** Compared to permissionless blockchains such as Bitcoin [1] and Ethereum [35], the existing BFT protocols generally face scalability issues. That is, they can only support a few hundred replicas in practice. The main reason is that the BFT protocol relies on majority voting among participants to achieve consensus, and the voting size<sup>4</sup> (the number of votes needed) grows linearly with  $n$ . Meanwhile, the communication and computing costs of the protocol also increase significantly as the voting size grows. Therefore, the most intuitive solution to this issue is to make the voting size (or QC size), relatively small even when  $n$  increases. The existing solutions can be categorized into *sharding* and *committee sampling*.

- **Sharding.** The sharding technique allocates replicas to different shards, each of which only processes the commands belonging to itself [22, 27]. In this way, the QC size only depends on the number of replicas contained in each shard, and therefore, this technique can increase the number of participating replicas to thousands. One major disadvantage of the sharding technique is its reliance on strong security assumptions. To see this, let us take a look at a most recent (generic) sharding consensus pattern, Kronos [26]. The construction of Kronos does not rely on any timing assumption. However, Kronos assumes that *each* shard satisfies, with probability 1, the condition that the proportion of Byzantine replicas is less than  $1/3$  (for partially synchronous and asynchronous networks) or  $1/2$  (for synchronous networks). This (strong) assumption, as shown in [37], can only be approximately satisfied if the number of replicas in each shard exceeds 100. In contrast, Kronos only runs experiments with 4-replica-sized shards. So the satisfaction of such an assumption cannot be guaranteed.
- **Committee Sampling.** Committee sampling is another solution to reducing the QC size. The basic idea is to select, from  $n$  replicas, a small number of replicas as the committee members. Now, this small committee runs the consensus protocol to reach a consensus result that is then conveyed to the remaining replicas. Therefore, the QC size only depends on the committee size instead of  $n$ , and thus, this technique can provide much better scalability. Committee sampling is the de facto model used in the scalable Byzantine

<sup>4</sup> In protocols that use threshold signatures to compress votes into quorum certificates (QC), the voting size is also called QC size.

agreement (BA) protocol. Starting from [19], most works focus on improving the communication complexity [2, 4] or round complexity [8] of the protocol under the assumption of an *ideal sampler*, which samples a small committee from all and ensures that the proportion of Byzantine replicas remains nearly the same. In particular, Algorand [13] uses the verifiable random function (VRF) to instantiate the ideal sampler under a partially synchronous network and introduces the notion of final and tentative consensus to handle the conflicting blocks caused by network asynchrony. Pando [34] introduces committee sampling in BFT for the first time and combines it with the decoupling method [10, 12] where transaction transmission and consensus are separated to enhance the throughput. Rather than serving as a generic framework, Pando constructs a concrete system using some key components elegantly. Furthermore, although the decoupling method can boost the system’s throughput by “balancing” all participants, it also introduces the duplicate packaging problem, leading to significant resource waste for the replicas.

**Importance of a scalability-enhanced framework.** Considering the enhancement of members’ scalability, it is worth noting that many BFT protocols have already been deployed [23, 25], which makes it quite expensive to replace existing protocols with new ones. In other words, a general and compatible framework that can safely extend the existing consensus protocol that supports 100-level replicas to support 1,000-level replicas is much more preferred in practice than deploying an entirely new approach. Although Kronos [26] provides a generic construction based on sharding, it suffers from unconvincing security analysis, as discussed before. Thus, there is still a lack of a secure and compatible scalability-enhanced framework for traditional BFT protocols. Therefore, we are devoted to finding a solution that serves as a general scalability-enhancing framework that can be applied as an extension to the widely used BFT protocols, including PBFT and HotStuff.

**Our approach.** To improve on the previous work, we focus on leveraging the committee sampling technique to BFT consensus protocols, achieving the QC size independent of  $n$ . We propose a novel module-based design that: (1) embeds the committee rotation into consensus so that the consistency of the consensus process can naturally guarantee the safety of committee rotation and (2) strictly separates the tasks of committee members from non-committee members so that non-committee replicas no longer need to be aware of the intermediate states of the consensus and thus reduce the communication complexity. We also provide rigorous proof of security for our design. Compared with the prior work on scalable BFT [34], our committee rotation requires confirmation through block delivery to ensure that each chosen committee is of a fixed size and is acknowledged by all replicas, even under network fluctuations.

### 1.1 Our Contribution

In this paper, we make the following contributions:

- **Generic scalability-enhancing framework for traditional BFT protocols.** We propose Walnut, the first universal committee-sampling-based

scalability-enhancing framework for traditional BFT protocols, which scales popular 1/3-fault tolerance BFT protocols in the partially synchronous network to support 1,000 replicas. For this framework, we define three modules and their interaction methods. To guarantee the safety and liveness during scaling, we abstract five properties required for the framework and prove that each instantiation satisfying them can scale the number of replicas the original protocol supports safely under our framework.

- **Concrete instantiation over HotStuff.** In order to demonstrate the usability of our framework, We provide a concrete instantiation by applying Walnut to HotStuff, named Walnut-HS. We prove that this instantiation satisfies both safety and liveness.
- **Large-scale deployment and performance evaluation.** We implement Walnut-HS on Amazon EC2 and evaluate its performance in two scenarios based on  $n$ . For  $400 \leq n \leq 1,000$ , the original HotStuff performs poorly or even breaks down, while Walnut-HS can work smoothly by changing the committee size. For instance, when the committee size is 200, our protocol can support 1,000 replicas with a slightly lower performance than the original HotStuff for 200 replicas. For  $n \leq 200$ , our framework can enhance the protocol performance. For example, when  $n = 100$ , by setting the committee size as 20, the peak throughput of Walnut-HS is 38.6% higher than HotStuff.

## 1.2 Comparison with Other Works

**Comparison with other VRF-based works.** Although Walnut, Pando [34], and Algorand [13] all use VRFs for committee sampling, Walnut differs from these works in the following respects. (1) Walnut is a *framework* that can be applied to many existing BFT protocols, whereas Pando and Algorand are specific protocols offering no generality. (2) These protocols use VRFs in different ways. Algorand is a PoS-type protocol in which a replica’s chance of joining the committee depends on its stake. In contrast, all replicas in Pando and Walnut have the same joining probability. Moreover, the committee size in Algorand and Pando can fluctuate, whereas Walnut fixes the size to guarantee its applicability to a broader range. (3) Pando and Walnut rely on smaller committees than Algorand. For example, with a failure probability of  $\delta = 10^{-9}$  and a resilience sacrifice of  $\epsilon = 0.12$  (which means 80% honest replicas), the committee sizes for Pando, Walnut, and Algorand are 200, 460, and 2000, respectively. Walnut’s committee is slightly larger than Pando’s because Walnut’s framework design makes its failure probability depend on the probability of having more than 1/3 Byzantine replicas in the committee, while Pando can more precisely analyze the failure probability at each protocol phase. We compare the performance of existing scalable BFT protocols in Table 1.

**Comparison with other scalability-enhancing methods.** Broadly, a consensus protocol’s scalability encompasses enhanced performance and expanded functionality. As noted in [3], techniques such as modifying communication topologies [14, 17, 31], using pipelining [16, 33], and incorporating suitable cryptographic primitives [6, 15, 20] can improve protocol efficiency, but their enhancement is limited. The core issue is that communication and computation costs still

Table 1: Performance of scalable BFT protocols.  $\epsilon$  is a small constant,  $L$  is the size of a block, and  $\kappa$  is the length of the security parameter. Our committee size is  $O(\kappa)$  following prior works. We assume that the QCs are instantiated through a set of digital signatures and consider the communication complexity for a single block. The communication complexity of Walnut-HS can be reduced to  $O(nL + \kappa^2 n)$  by setting the frequency of committee rotation to  $1/\kappa$ .

| Protocols  | Generic Framework | Resilience              | Communication Complexity         |
|------------|-------------------|-------------------------|----------------------------------|
| Pando [34] | No                | $f < (1/3 - \epsilon)n$ | $O(nL + \kappa nL + \kappa^2 n)$ |
| Walnut     | Yes               | $f < (1/3 - \epsilon)n$ | $\swarrow$                       |
| Walnut-HS  | $\swarrow$        | $f < (1/3 - \epsilon)n$ | $O(nL + \kappa^2 n + \kappa^4)$  |

grow with  $n$ . Nevertheless, while our main objective is to increase the number of replicas supported by the BFT protocol, our method also boosts performance in smaller-scale networks.

For functionality expansion, current research mainly focuses on dynamicity and order-fairness. Dynamic BFT protocols [11, 29, 38] consider the scenario with dynamic participation, allowing replicas to freely join or leave the protocol without affecting protocol execution. On the other hand, order-fairness [18, 30] aims to ensure that the final order of transactions reflects the order in which they arrived in the network, preventing malicious replicas from manipulating transaction order to gain undue advantages.

## 2 System Model and Building Blocks

### 2.1 System Model

**BFT.** The Byzantine fault-tolerant (BFT) protocol is a long-studied topic. In a BFT protocol, clients submit commands while replicas package them into blocks and consistently deliver them. A BFT protocol with  $n$  replicas tolerating up to  $f$  Byzantine faults achieves “optimal resilience” if  $f < n/3$ . In this work, we adopt the “near-optimal resilience” instead, following the way of thoughts in works focusing on the scalability of BFT protocols such as Pando [34] and scalable Byzantine agreement [4, 13]. That is, we only assume that  $f < (1/3 - \epsilon)n$  holds, where  $\epsilon$  is a constant value and  $0 < \epsilon < 1/3$ .

A BFT protocol is secure if it satisfies the following two properties.

- **Safety:** If a correct replica delivers a command  $cmd$  before delivering  $cmd'$ , then no correct replica delivers a command  $cmd'$  without first delivering  $cmd$ .
- **Liveness:** If a command  $cmd$  is submitted to all correct replicas, then all correct replicas eventually deliver  $cmd$ .

In addition, since we target the BFT protocols that have been widely deployed in real-world applications such as Cosmos [23] and Libra [25], we use *traditional* BFT to represent the leader-based and non-decoupled BFT protocol.

**Committee-sampling-based BFT.** The committee-sampling-based BFT is a special type of BFT. Roughly, to reach consensus among  $n$  replicas, this method samples  $\lambda = O(\kappa)$  out of  $n$  replicas to form a committee that is responsible

for achieving consensus, where  $\kappa$  is the length of the security parameter. For simplicity, we assume  $\lambda = \kappa$  from here on.

In practice, a constantly rotated committee is preferred, especially when considering the non-static adversaries. Thus, we use  $\mathcal{C}_c$  to denote the  $c$ -th committee (i.e.,  $c$  is the committee number). Assuming the number of faulty replicas in the first committee is  $t_0 < \lambda/3$ , we need to ensure that, except with negligible probability  $\text{negl}(\kappa)$ , the number of faulty replicas in each subsequent committee is  $t_c < \lambda/3$  ( $c \geq 1$ ). In other words, we use  $t$  to denote the maximum number of faulty replicas in each committee, i.e.,  $t = \lfloor \frac{\lambda-1}{3} \rfloor$ . Thus, a Byzantine quorum consists of  $\lceil \frac{\lambda+t+1}{2} \rceil$  replicas. When  $\lambda = 3t + 1$ , a *quorum certificate* (QC) consists of  $\lambda - t$  signatures generated by a Byzantine quorum.

Since the number of faulty replicas in the  $c$ -th committee may break the  $\lambda/3$  bound with a negligible probability  $\text{negl}(\kappa)$ , the security definition of the whole protocol needs to be slightly modified. Concretely, the protocol should satisfy safety and liveness with probability  $1 - \text{negl}(\kappa)$ .

We emphasize that, as far as we know, Pando is the only known committee-sampling-based BFT protocol, even though the committee-sampling methodology has been used in other kinds of protocols.

**Network model.** We consider a partially synchronous network where there exists an unknown Global Stabilization Time (GST) and a known bound  $\Delta$ , such that after GST, all messages among honest replicas will arrive within  $\Delta$ . The adversary can delay or reorder the messages. In addition, we assume "atomic sends" similar to [34] where an honest replica  $\mathcal{P}_i$  can send a message to multiple replicas, and the adversary can control the network of  $\mathcal{P}_i$  either before or after it sends the messages to all receivers.

**Adversary model.** We consider the (weakly) adaptive adversary that can selectively corrupt replicas by sending an instruction as long as the fraction of faulty replicas is less than  $1/3 - \epsilon$ . When correct replicas receive such an instruction, it takes at least  $\tau$  time before they are corrupted. The  $\tau$  time is necessary for committee-based protocols [21, 32] to ensure: (1) Correct replicas have left the committee before being corrupted; (2) All correct replicas have delivered the highest block that a correct replica  $\mathcal{P}_i$  has delivered before  $\mathcal{P}_i$  is being corrupted.

The first restriction is to limit the corruption ability of the adversary. In fact, what we want is that the adversary must decide which replicas to corrupt in a committee before the committee is formed. After that, until the next committee starts working, the adversary cannot change the set of corrupted replicas. The second is to ensure our catch-up mechanism will work under the partially synchronous network. In other words,  $\mathcal{P}_i$  always helps others to synchronize the block until the block is delivered by all correct replicas. Synchronous protocols naturally satisfy this restriction since all messages arrive within a known bound.

## 2.2 Building Blocks

**Verifiable random function (VRF).** In [28], Micali introduced the concept of verifiable random function (VRF), and later, they used VRF as the randomness source for the committee sampling to construct Algorand [13].

Assume each replica  $\mathcal{P}_i$  hold a pair of public and private keys  $(pk_i, sk_i)$ , where  $pk_i$  is known by all replicas and  $sk_i$  is held secretly.  $\mathcal{P}_i$  can use its private key to generate a random number  $\rho$  with a proof  $\pi$  on a message  $m$ , i.e.,  $(\rho, \pi) \leftarrow \text{VRFGGen}_{sk_i}(m)$ . Each replica can use the public key of  $\mathcal{P}_i$  to verify the validity of the random number, i.e.,  $0/1 \leftarrow \text{VRFVrfy}_{pk_i}(\rho, \pi, m)$ . The random number  $\rho$  is valid if 1 is returned and invalid otherwise.

In this work, we also use VRF to implement the committee sampling function. Specifically,  $m$  includes a view number  $v$  and a committee number  $c$ . Replicas in view  $v$  call  $\text{VRFGGen}()$  function to generate  $(\rho_i, \pi_i)$  pairs. If  $\rho_i$  is smaller than a difficulty number  $D$ , then  $\mathcal{P}_i$  is a candidate for the next committee (i.e.,  $\mathcal{C}_{c+1}$ ). The difficulty number  $D$  ensures that the expected number of replicas who hold valid  $(\rho_i, \pi_i)$  pairs is  $\lambda$  in each view. For example, when  $n = 500$ ,  $\lambda = 100$ , we set  $D = 0.2 \times E$  where  $E$  denotes the space size of the random number  $\rho$ . Thus, the probability that each replica is selected as a candidate for the next committee is 0.2, deriving from the randomness of  $\rho_i$ . We emphasize that, in Walnut, every replica has the same probability of joining the committee, whereas in Algorand, this probability depends on the replica's stake.

### 3 The Walnut Framework

This section illustrates how Walnut works as a generic framework, including the security guarantee.

#### 3.1 Motivation

Traditional BFT protocols such as PBFT [7] and Hotstuff [36] only perform well with around 100 replicas due to the high communication and computational cost that increases as  $n$  grows (From here on, we refer to the traditional/original BFT protocol as  $\Pi_{\text{BFT}}$ ). Pando [34] is the first practical and concrete BFT that scales to about 1,000 replicas using committee sampling. A constant committee rotation is used to resist adaptive adversaries. Due to the inherent randomness of the VRF mechanism, the committee size in Pando naturally fluctuates. Furthermore, as discussed in Section 1, this work does not aim to propose a specific BFT protocol but rather focuses on improving the scalability of existing solutions. To this end, we aim to fix the committee size to ensure broader applicability across various protocols and to mitigate potential security risks.

To achieve the fixed-size committee sampling, our key observation is that the universally acknowledged committee of a fixed size aligns perfectly with the functionality of consensus protocols. Thus, our solution is to elegantly embed the election for committee members into the BFT protocol, which leads to our Walnut. It is worth noting that instead of proposing a new and isolated BFT consensus, our way of thinking works as a framework for enhancing the scalability of the existing BFT protocols.

Moreover, to reduce communication costs further, Walnut strictly differentiates the workflow between committee and non-committee members so that

non-committee members no longer need to process the intermediate states in consensus. Meanwhile, we carefully design our synchronization mechanisms to handle potential network asynchrony.

### 3.2 Overview

To begin with, we provide a high-level description of Walnut. Generally, Walnut consists of three interactive modules and outputs a single chain that determines the total order of all blocks, as shown in Figure 1.

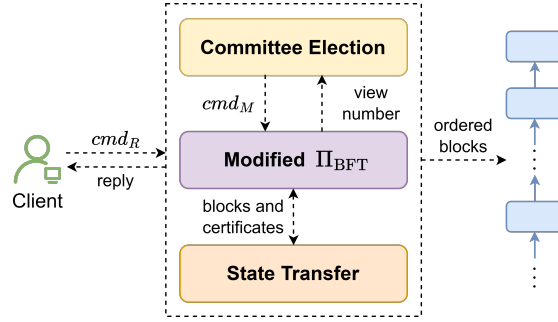


Fig. 1: Walnut architecture. The black dashed arrows represent the transmission of messages, while the blue solid arrows denote the reference relationships between blocks.

Roughly, at any given time, there are two types of replicas in Walnut, i.e., committee members and non-committee members, and each is responsible for different tasks. Committee members run the *modified*  $\Pi_{\text{BFT}}$  module to package commands, deliver blocks, and convey blocks to non-committee members ( $\Pi_{\text{BFT}}$  represents the original BFT protocol). Meanwhile, non-committee members simply deliver blocks by the *State Transfer* module with the committee instead of being aware of any intermediate results. Besides, all replicas run the *Committee Election* module to participate in committee rotation.

**Committee election module.** The committee election module is view-based, and each replica generates a random number in each view. Those with the smallest  $\lambda$  numbers are candidates for the next committee, and their numbers will be packaged by the leader in the modified  $\Pi_{\text{BFT}}$  into a block. Once the block is delivered, the committee will change, and these replicas will take over the work as new committee members.

**Modified  $\Pi_{\text{BFT}}$  module.** The current committee members run the *modified*  $\Pi_{\text{BFT}}$  module (M- $\Pi_{\text{BFT}}$  for short). Compared to the original  $\Pi_{\text{BFT}}$  protocol, slight modifications are introduced in M- $\Pi_{\text{BFT}}$ , to get in accordance with the whole framework. Specifically, to support committee rotation, we treat the random number used for committee election as a new type of command (e.g., the *membership command*  $\text{cmd}_M$ ), in distinguishing from the other commands dealt in the original  $\Pi_{\text{BFT}}$  (e.g., the *regular command*  $\text{cmd}_R$ ). Notably, in this work, a valid block contains either  $\lambda$  or 0  $\text{cmd}_M$ . Thus we sometimes abuse the notation



of  $cmd_M$  to denote a set of  $\lambda$  membership commands. When a block is delivered, each replica will execute all the regular commands and send the results to the corresponding client, just as they do in traditional BFT protocols. What is different is that if the block contains exactly  $\lambda$  membership commands, then committee rotation is triggered.

In addition, since the committee keeps changing, the block verification procedure needs to be modified accordingly. Moreover, we carefully design synchronization mechanisms for different situations to handle network fluctuation, both for replicas in the current and the next committees. It is worth mentioning that for non-committee replicas, the state transfer module can be viewed as their synchronization mechanism.

**State transfer module.** This module mainly aims at synchronizing states. Two types of replicas need to run this module: (1) Non-committee replicas and (2) Replicas that are in the next committee. The former ones run this module to synchronize delivered blocks from the committee. The latter ones run this module to synchronize the latest and necessary information from the current committee for subsequent  $M-\Pi_{BFT}$  execution. This is to ensure that the states of all correct replicas in the next committee are the latest despite the impact of network asynchrony. Note that a replica can belong to both types simultaneously.

Subsequently, we state each module separately.

### 3.3 Committee Election

As mentioned in Section 3.2, the BFT consensus chain is maintained by a rotating committee. An ongoing (and rotatable) committee challenges not only security but also the decentralization of the system as a whole. Our solution is to embed committee rotation into block delivery so that all replicas can perceive the rotation and verify the validity of each committee. Specifically, to achieve the fixed-size and rotatable committee election, we illustrate how replicas participate in the committee and the committee rotation method. These methods ensure that the cardinality of each committee is accurate.

**Committee participation.** Let  $\lambda = 3t + 1$  be the committee size. The first  $\lambda$  committee members, which contain  $t$  Byzantine replicas at most, could be hardcoded into the genesis block. We omit details of selecting such replicas. Then, in each view, each replica calls the  $VRFGen()$  function to get a random number  $\rho$  and a corresponding proof  $\pi$ , forming a  $cmd_M$ . If the  $\rho$  is valid (e.g., smaller than a number  $D$  that we predefined), the replica will send its  $cmd_M$  to the leader in the  $M-\Pi_{BFT}$ . Using VRF ensures that all replicas in the system can publicly verify the generated random numbers and proofs (or  $cmd_M$ ).

By running  $M-\Pi_{BFT}$ , replicas in  $\mathcal{C}_c$  can collect  $cmd_R$  and  $cmd_M$  in each view. Once the leader collects enough membership commands in a view, it will select the smallest  $\lambda$  and package them into the proposed block. When the block is delivered, committee rotation is triggered.

**Committee rotation.** Each replica records the current (e.g., the  $c$ -th) committee by  $\mathcal{C}_c$ , and the leader in each view collects  $cmd_M$  from all replicas. When

the most recent block delivered contains exactly  $\lambda$  membership commands, correct replicas in  $\mathcal{C}_c$  will help others synchronize the block and stop working. All correct replicas will finally deliver this block, perceive the committee rotation, and update  $\mathcal{C}_c$  to the next one. The  $\lambda$  corresponding replicas form the next (e.g.,  $(c+1)$ -th) committee  $\mathcal{C}_{c+1}$  and run  $\text{M-}\Pi_{\text{BFT}}$  to deliver new blocks as the last committee did.

Figure 2 shows an example for  $n = 20$  and  $\lambda = 4$ .  $\mathcal{C}_c$  runs the  $\text{M-}\Pi_{\text{BFT}}$  to deliver blocks. In view  $v$ , each replica  $\mathcal{P}_i$  generates a  $\rho_i$  and its corresponding proof  $\pi_i$ . However, only the random numbers generated by  $\mathcal{P}_{16}, \mathcal{P}_{17}, \mathcal{P}_{18}, \mathcal{P}_{19}, \mathcal{P}_{20}$  are valid (i.e., smaller than  $D$ ). These replicas send their  $\text{cmd}_M$  to the leader  $\mathcal{P}_1$ .  $\mathcal{P}_1$  then collects five  $\text{cmd}_M$  in total and selects the smallest four from them (i.e.,  $\mathcal{P}_{17}, \mathcal{P}_{18}, \mathcal{P}_{19}, \mathcal{P}_{20}$  are selected). These commands are packaged in the block proposed by  $\mathcal{P}_1$  in view  $v$ . Once the block is delivered, committee rotation happens and  $\mathcal{P}_{17}, \mathcal{P}_{18}, \mathcal{P}_{19}, \mathcal{P}_{20}$  will take over the work as members in  $\mathcal{C}_{c+1}$ .

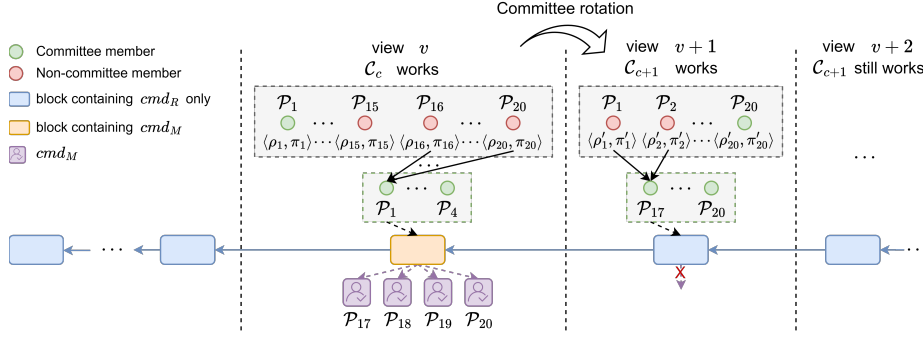


Fig. 2: An example of the committee rotation for  $n = 20$  and  $\lambda = 4$ . The gray dashed boxes represent the set of all replicas, while the green ones denote the set of committee members. The black solid arrows indicate message transmission. The black dashed arrows represent the packaging of commands into a block. The blue solid arrows denote the reference relationships between blocks. The purple dashed arrows represent the reference relationships between blocks and  $\text{cmd}_M$ .

**Fixed committee size.** The randomness of VRF implies that it is not necessarily possible to generate precisely  $\lambda$  valid numbers (i.e., less than  $D$ ) in each view. We fix the problem with a careful design. In each view, denote the total number of valid numbers submitted to the committee by  $\lambda'$ . If  $\lambda' > \lambda$ , the current  $\text{M-}\Pi_{\text{BFT}}$  committee simply selects the smallest  $\lambda$  numbers from all (e.g., see the block proposed by  $\mathcal{P}_1$  in view  $v$  in Figure 2). If  $\lambda' < \lambda$ , then new blocks will still be delivered, but no committee rotation happens (e.g., see the block proposed by  $\mathcal{P}_{17}$  in view  $v+1$  in Figure 2). It is worth noting that, in the latter case, replicas need to refresh the random numbers generated by VRF in subsequent views to make sure that enough  $\text{cmd}_M$  will be collected in some future view, and thus, committee rotation can be triggered finally. The concrete solution is to use the view number of  $\text{M-}\Pi_{\text{BFT}}$  as a seed for the  $\text{VRFGen}()$ .

This design, while intuitive, ensures two critical requirements for the scalability-enhancing BFT effectively: (1) the selected committee can continuously rotate, and (2) each committee consists of exactly  $\lambda$  replicas, which are universally acknowledged. In our design, the fixed-size committee rotation is embedded into the block delivery.

### 3.4 Modified BFT Protocol $M\text{-}\Pi_{\text{BFT}}$

As mentioned, each committee runs  $M\text{-}\Pi_{\text{BFT}}$  to generate blocks. The workflow of  $M\text{-}\Pi_{\text{BFT}}$  is essentially the same as the original  $\Pi_{\text{BFT}}$  except for some minor and necessary modifications for supporting the committee rotation. Roughly, three kinds of modifications are needed: (1) block construction and verification, (2) block delivery, and (3) catch-up mechanism. Since the detailed modifications depend on the specific structure of  $\Pi_{\text{BFT}}$ , we only give qualitative descriptions of the modifications as below and provide a specific instance using HotStuff [36] in Section 4 and Appendix B, where all the modifications are marked in purple.

**Block construction and verification.** The block construction of  $M\text{-}\Pi_{\text{BFT}}$  differs from the original  $\Pi_{\text{BFT}}$  by subdividing commands contained in blocks into the regular commands (i.e., the original  $\Pi_{\text{BFT}}$ 's commands) and membership commands (i.e., introduced in committee election).

Correspondingly, in  $M\text{-}\Pi_{\text{BFT}}$ 's block verification procedure, in addition to verifying the original  $cmd_R$  as  $\Pi_{\text{BFT}}$  does,  $M\text{-}\Pi_{\text{BFT}}$  verifies  $cmd_M$  via the VRF-verification. Moreover, since the committee is rotatable, replicas need to distinguish two cases when verifying blocks: (1) the same committee proposes the block as that of its parent, and (2) a new committee proposes the block. For the former, the verification is identical to the original protocol except for verifying the membership commands. For the latter, the replicas need to utilize information (e.g., some quorum certificates) from the last committee to ensure that the block extends the latest delivered block without causing conflict.

**Block delivery.** When delivering a block, similar to the original  $\Pi_{\text{BFT}}$ , all replicas should execute all regular commands in the block and send the execution result to the corresponding client as a reply. The main difference is that once this block has  $\lambda$  membership commands, all replicas update the committee from the current  $\mathcal{C}_c$  to the new  $\mathcal{C}_{c+1}$ . Moreover, after the delivery of a block, committee members need to transfer the block to all non-committee members. If committee rotation happens, they will additionally help members in the next committee to synchronize some necessary and latest information for running  $M\text{-}\Pi_{\text{BFT}}$ .

**Catch-up mechanism.** Since we assume a partially synchronous network, another challenge is that some replicas may miss some blocks due to network fluctuation. In  $\Pi_{\text{BFT}}$ , these lagging replicas can simply synchronize with others since all replicas are members of a fixed committee. This method, however, is not applicable in  $M\text{-}\Pi_{\text{BFT}}$  since the committee continuously rotates.

For this issue, we present a catch-up mechanism where each committee  $\mathcal{C}_c$  is responsible for synchronizing all blocks delivered by  $\mathcal{C}_c$ . The core idea is quite intuitive: Once a replica experiences a repeated timeout, it will seek help from

the current committee it records, and the committee members will synchronize all blocks delivered by the committee along with relevant proofs to it.

### 3.5 State Transfer

The state transfer process is responsible for synchronizing information between committee and non-committee replicas as well as between the current committee and the next committee during committee rotations.

**Synchronization between committee and non-committee replicas.** The committee sampling mechanism inherently requires the committee to transfer all delivered blocks to all non-committee replicas. We let the leader in each view of  $M-\Pi_{\text{BFT}}$  transfer the block to all replicas when proposing it. After the delivery of a block, all committee members should send the proof of delivery for the block to all non-committee replicas. Once a non-committee replica collects enough proofs, it will deliver the corresponding block.

**Synchronization across committees.** When committee rotation occurs, due to network fluctuation, it is possible that some replicas belonging to the next committee do not have the most up-to-date state required to run  $M-\Pi_{\text{BFT}}$ . Therefore, whenever committee rotation happens, we let members of the next committee synchronize with the current committee to get the latest information.

### 3.6 Properties Required for Each Module

Working as a generic framework to enhance the scalability of the underlying protocol  $\Pi_{\text{BFT}}$ , each concrete instantiation of Walnut characterizes with distinctive modifications for different  $\Pi_{\text{BFT}}$ . Meanwhile, the scalability-enhanced BFT protocol should achieve security similar to the original BFT protocol, as we have formalized in Section 2. Here, we introduce the requirements for each module.

**Intuitive requirements.** Informally, to satisfy safety and liveness, we first summarize three other requirements of Walnut in addition to the security of the original  $\Pi_{\text{BFT}}$ . Intuitively, these requirements can be viewed as the gap between the traditional  $\Pi_{\text{BFT}}$  with the fixed committee and our Walnut with the rotating committee. In fact, each requirement corresponds to a module in our framework.

(1) *Randomness of elected committees.* One challenge in committee election is that if the adversary knows in advance which replicas are part of the next or a later committee, it can plan attacks on that committee beforehand, endangering the protocol's security. Hence, randomness must be ensured so that nobody can predict new committee members ahead of time.

(2) *Safety and liveness across committees.* Safety and liveness within the same committee are guaranteed by the original BFT protocol  $\Pi_{\text{BFT}}$ . However, when the committee changes, i) for safety, we need to ensure that the new committee will acknowledge all previously delivered blocks and propose new blocks that extend this public chain, and ii) for liveness, we need to ensure that the new committee will deliver new blocks and make progress.

(3) *State synchronization.* As mentioned above, the main function of state transfer is to help non-committee replicas synchronize blocks with the committee.

Since non-committee replicas are unaware of the intermediate states of  $M\text{-}\Pi_{\text{BFT}}$ , they can deliver a new block if and only if a correct replica in the committee delivers the block.

**Formal definitions of all requirements.** Based on the observations above, we formally define five properties. Two of them originate from the original BFT, and the other three are our additional requirements. In a nutshell, the same-com safety, same-com liveness, and smooth rotation properties are required for the  $M\text{-}\Pi_{\text{BFT}}$  module, while the random election is for the committee election module, and the state synchronization is for the state transfer module. To distinguish blocks proposed and delivered by different committees, we say that a block  $b$  is related to the  $c$ -th committee if  $b$  is proposed by a correct replica in  $\mathcal{C}_c$ . When we say  $b'$  directly extends  $b$ , it means  $b$  is the parent of  $b'$ .

- **Property 1.** (Random Election). If  $t_c < \lambda/3$  holds for the current committee  $\mathcal{C}_c$  and a correct replica in  $\mathcal{C}_c$  delivers a block  $b$  containing a  $cmd_M$ , then the corresponding  $\lambda$  replicas referred to by  $cmd_M$  can be viewed as randomly selected from the system.
- **Property 2.** (Same-Com Safety). If  $t_c < \lambda/3$  holds for the current committee  $\mathcal{C}_c$  and a correct replica in  $\mathcal{C}_c$  delivers a command  $cmd$  before delivering  $cmd'$ , then no correct replica in  $\mathcal{C}_c$  delivers  $cmd'$  without first delivering  $cmd$ .
- **Property 3.** (Same-Com Liveness). If  $t_c < \lambda/3$  holds for the current committee  $\mathcal{C}_c$ , and the committee does not change, once a command  $cmd$  is submitted to all correct replicas in  $\mathcal{C}_c$ , all correct replicas in  $\mathcal{C}_c$  eventually deliver  $cmd$ .
- **Property 4.** (Smooth Rotation). If  $t_c < \lambda/3$  holds for the committee  $\mathcal{C}_c$  and a correct replica  $\mathcal{P}_i$  in  $\mathcal{C}_c$  delivers a block  $b$ , then all correct replicas in  $\mathcal{C}_c$  eventually deliver  $b$ . If  $b$  contains a  $cmd_M$ , then  $b$  is the last block delivered by the  $c$ -th committee. Moreover, If  $t_{c+1} < \lambda/3$  holds for the committee  $\mathcal{C}_{c+1}$ , a correct replica  $\mathcal{P}_j$  in  $\mathcal{C}_{c+1}$  eventually delivers some new valid blocks and the first block it delivers directly extends  $b$ .
- **Property 5.** (State Synchronization).
  - i) If  $t_c < \lambda/3$  holds for the committee  $\mathcal{C}_c$  and a correct replica in  $\mathcal{C}_c$  delivers a block  $b$  in view  $v$ , then all correct replicas in the system that are not in  $\mathcal{C}_c$  eventually deliver  $b$  in view  $v$ .
  - ii) If  $t_c < \lambda/3$  holds for the committee  $\mathcal{C}_c$  and a correct replica in the system delivers a block  $b$  in view  $v$  related to the  $c$ -th committee, then at least one correct replica in  $\mathcal{C}_c$  has delivered  $b$  in view  $v$ .

**Security proof.** We state that the above five properties are sufficient to ensure overall safety and liveness with the help of Chernoff Upper Tail Bound [9]. The sketch and details of the proof is given in Appendix A.

## 4 The Walnut-HS Instantiated by HotStuff

To verify the effectiveness of Walnut, we apply this framework to HotStuff [36], aiming to extend the number of supported replicas to a thousand. Specifically, we use the basic HotStuff (denoted as  $\Pi_{\text{HS}}$ ) as the  $\Pi_{\text{BFT}}$  protocol in our instantiation. To fit into our framework, we instantiate each module. Roughly,

- The data structure of  $\Pi_{\text{HS}}$  needs to be slightly modified to support the committee rotation. More specifically, blocks in our system contain  $cmd_R$  as well as  $cmd_M$ . Besides, we need to record the current committee number in the blocks and the quorum certificates.
- The committee election process is view-based, where each replica calculates a random number and a corresponding proof in each view. A new view starts when a replica receives  $\lambda - t$  NEW-VRF-VIEW messages from the current committee running  $\text{M-}\Pi_{\text{HS}}$ . Membership commands are formed and sent to the current committee in each view.
- The  $\text{M-}\Pi_{\text{HS}}$  process is also view-based. In each view, a designated leader collects  $cmd_R$  and  $cmd_M$  and proposes both kinds of commands in a block.
- After the committee reaches an agreement, they will send a state transfer message to non-committee replicas, and finally, all replicas obtain the block and deliver it. If the delivered block contains  $\lambda$  membership commands, then committee rotation is triggered, and the current committee additionally transfers some QCs to the next committee.

Due to space constraints, the detailed description and analysis of Walnut-HS are provided in Appendix B, while the security proof is presented in Appendix C.

## 5 Failure Probability, Resilience, and Committee Size

We analyze the probability of safety and liveness violation of Walnut in Appendix A. The result shows that Walnut achieves safety and liveness with probability  $1 - \text{negl}(\kappa)$ .

To better understand the relationship between the failure probability  $\delta$ , the resilience sacrifice  $\epsilon$ , and the committee size  $\lambda$ , we first clarify how our failure probability is calculated. By Lemma 1 in Appendix A, if  $t_{c-1} \leq \lambda/3$ , then  $t_c \leq \lambda/3$  holds with probability at least  $1 - \delta$ . Following the approach outlined in Pando [34], we model the committee election process as a repeated Bernoulli experiment. Thus, the failure probability can be calculated by binomial distribution given a specific  $\epsilon$  and  $\lambda$ . For example, when  $\delta = 10^{-4}$ ,  $\lambda = 200$ , by binomial distribution, we know that  $\Pr[Y \geq 67] = 0.0000993 < \delta$  when  $\epsilon = 0.115$ , where  $Y$  denotes the number of faulty replicas in a committee.

Next, we will demonstrate that our protocol is highly practical. I.e., when supporting a large number of replicas (e.g., 1000), the protocol can maintain good resilience while keeping the committee size relatively small (thus practical).

**Resilience vs. committee size under different failure probability.** In Figure 3, we demonstrate the relationship between  $\lambda$  and  $\epsilon$  given different  $\delta$ . To achieve the same level of security, the resilience sacrifice  $\epsilon$  decreases as  $\lambda$  increases. Actually, when  $\delta = 10^{-4}$ , for a committee size  $\lambda \geq 160$ , the resilience of the whole system is between  $1/5$  and  $1/3$ .

**Failure probability, resilience, and committee size for large scales.** From the above analysis, it is easy to see that, given a desired  $\delta$  and  $\lambda$ , the resilience sacrifice  $\epsilon$  can be directly calculated by binomial distribution. In other words,  $\epsilon$  is

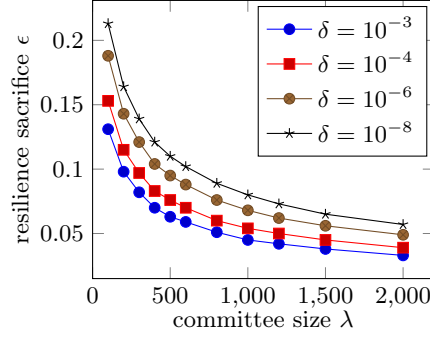


Fig. 3:  $\lambda$  vs.  $\epsilon$  to satisfy different failure probabilities desired.

only related to  $\delta$  and  $\lambda$ , and is totally independent of  $n$ . Therefore, theoretically, the number of replicas the system can support can be arbitrarily large, while the values of  $\delta$ ,  $\epsilon$ , and  $\lambda$  remain unchanged. So, the results we computed earlier ( $\delta=10^{-4}$ ,  $\lambda \geq 160$ ,  $1/3 - \epsilon \in [1/5, 1/3]$ ) apply for  $n=500, 1000, 2000$  and so on.

**Committee rotation frequency and the failure probability.** All the previous calculations regarding failure probability were made under the assumption that a committee election occurs for every view. However, in practice, setting a committee rotation frequency is a more common approach. For example, the committee may rotate every 24 hours. Under such a configuration, given an average protocol latency of 5 seconds, the committee election would be triggered once every 17,280 views on average. In this case, our failure probability would be  $10^{-4}$  times the values presented in Figure 3.

## 6 Implementation and Evaluation

### 6.1 Implementation

We implement Walnut-HS and HotStuff in Golang using the same library. We compare these two protocols to validate the effectiveness of our framework. Our implementation involves about 12,000 LOC for the protocols and around 1,900 LOC for evaluation. We use TCP long connections to achieve reliable point-to-point channels and use the elliptic curve P-256 for digital signatures. For VRF, we instantiate our `VRFGen()` and `VRFVrfy()` functions using an open-source library<sup>5</sup> based on Golang. To support open science and further research, Walnut-HS is publicly available at <https://git.chainweaver.org.cn/Flq/walnut-hs>.

We evaluate the performance of the protocols through experiments on Amazon EC2 using *m5.xlarge* instances (instance for short). They provide four virtual CPUs and 16GB memory. For experiments with up to 100 replicas, we run one replica on each instance. For larger-scale experiments, ten replicas are run on each instance. Our experiments are all conducted in the WAN setting. We evaluate the latency and throughput of our protocol under different network sizes  $n$

<sup>5</sup> <https://github.com/yoseplee/vrf>

and batch sizes  $b$ . The command size is 250 bytes by default, which is the size of a typical Bitcoin transaction.

Following the experiment strategies adopted in [34], when evaluating Walnut-HS, we also conduct experiments with different committee sizes. We use Walnut-HS ( $x$ ) to denote the experiment where the committee size is  $x \times n$ .

## 6.2 Evaluation Methodology

We evaluate the usability of Walnut-HS in traditional and large-scale network sizes and compare the results with Hotstuff. The parameter  $x$  varies from 0.2 to 1 to assess different committee sizes in Walnut-HS.

- **The traditional size:**  $n = 100, 200$ . For Walnut-HS ( $x$ ) and Hotstuff, we evaluate: (1) The latency vs. throughput, to find the peak throughput for each<sup>6</sup>. (2) The latency for  $b = 1,000$  and  $b = 5,000$ , to compare their performance. (3) The Latency of different commands in Walnut-HS ( $x$ )<sup>7</sup>, to evaluate the impact of our new command  $cmd_M$ .
- **The large-scale size:**  $n$  varies from 200 to 1,000. Since we find that HotStuff performs too poorly to be acceptable as  $n > 200$ , we focus on evaluating Walnut-HS with fixed committee size under different  $n$ , with batch size  $b = 1000, 5000, 10000$  respectively.

## 6.3 Evaluation Results

**Summary of results.** We summarize our experiment results as below.

- **Scalability with increased  $n$ .** For Hotstuff, although we successfully run it using up to 400 replicas, we are not able to conduct experiments on a larger scale. Besides, Hotstuff performs poorly when  $n > 200$ , e.g., the latency is around 7.9s for  $n = 400$  and  $b = 5,000$ . Contrarily, Walnut-HS easily supports up to 1,000 replicas, with only a slight performance degradation as  $n$  grows.
- **Performance under the same  $n$ .** Under the same network size  $n$ , Walnut-HS (1) exhibits slightly lower overall performance than HotStuff. However, for  $x < 1$ , i.e., smaller committee size, Walnut-HS sees a significant performance boost. For example, when  $n = 100$ , Walnut-HS (0.2)’s peak throughput is 38.6% higher than HotStuff.
- **Impact of introducing  $cmd_M$ .** For each  $n$  and  $x$ , the latency of  $cmd_M$  is always slightly higher than that of  $cmd_R$ . For instance, for  $n = 200$  and  $b = 5,000$ , the latency of  $cmd_M$  is 9.3% higher than  $cmd_R$  in Walnut-HS (0.6).

**Detailed Results.** We now detail the results summarized above and provide more performance analysis.

**(1) Scalability of Walnut-HS.** We report the performance of Walnut-HS for  $n = 200$  to 1,000 in Figure 4a under  $b = 1,000$ ,  $b = 5,000$  and  $b = 10,000$ ,

<sup>6</sup> The experiment is only conducted for  $n = 100$  since HotStuff behaves poorly when  $n = 200$ .

<sup>7</sup> Note that  $x = 1$  is unnecessary since there is no committee rotation in this case.



respectively. In all of these experiments, the committee size of Walnut-HS is 200. Thus, the parameter  $x$  is varied for different network sizes. The corresponding throughput is shown in Appendix D. We also compare the performance with HotStuff and find the following facts.

- Generally, the latency of the two protocols grows as  $n$  grows, but at different rates. The latency of HotStuff ( $n=400$ ) is more than three times higher than HotStuff ( $n=200$ ), and we could not run the experiments with more replicas. In contrast, Walnut-HS can easily support up to 1,000 replicas using the same instances, with slight performance degradation. For example, The latency of Walnut-HS ( $n=1,000$ ) is only 28.2% higher than that of Walnut-HS ( $n=200$ ) and 94.9% higher than that of HotStuff ( $n=200$ ). This is all thanks to the relatively small committee size.
- An interesting finding is that when  $b=1,000$  and  $b=5,000$ , the latency of Walnut-HS for  $n=400$  is lower than  $n=200$ , even though the two protocols have the same committee size. This is mainly due to load dispersion. More specifically, when  $n=200$ , all replicas are committee members. Thus, each instance needs to handle the work of 10 committee members. When  $n=400$ , although the committee size remains the same, the committee members are evenly distributed among 40 instances. Each instance only needs to handle the work of 5 committee members on average, leading to better performance.
- However, for  $b=10,000$  or for larger-scale experiments ( $n \geq 600$ ), the latency grows as  $n$  grows. This is because our experiments are conducted with limited bandwidth; thus, in each view, the phase that the leader broadcasting blocks becomes a bottleneck. For instance, in the experiment with  $n=1,000$  and  $b=10,000$ , the amount of data the leader needs to broadcast in each view is about 2GB. We deduce that the bottleneck of block transmission on individual replicas can be significantly reduced by introducing the classic gossiping protocol [17] like in [22, 27]. Thus, it is promising that, with the same committee size, the performance of Walnut-HS will improve as  $n$  grows.

**(2) Latency vs. throughput of Walnut-HS and HotStuff.** We evaluate the latency vs. throughput of Walnut-HS and HotStuff. Although we could run HotStuff with up to 400 replicas, the throughput for  $n=200$ ,  $n=300$ , and  $n=400$  is very low and thus omitted from our comparisons. We conclude this is mainly because the CPU and bandwidth of the instances are not sufficient to support such high computational and communication overhead.

We show the experiment results for  $n=100$  in Figure 4b. Our results show that the peak throughput of Walnut-HS (1) is only slightly lower than HotStuff due to the additional VRF cost. However, the performance of Walnut-HS can be significantly improved by reducing the committee size. The smaller the committee size, the better the performance. Both Walnut-HS (0.8) and the other experiments with smaller committee sizes achieve significantly better performance than HotStuff. For example, the peak throughput of Walnut-HS (0.4) and Walnut-HS (0.2) is 16.1% and 38.6% higher than HotStuff, respectively.

**(3) Latency of Walnut-HS and HotStuff with different network sizes.** We assess the latency of Walnut-HS with different committee sizes and compare

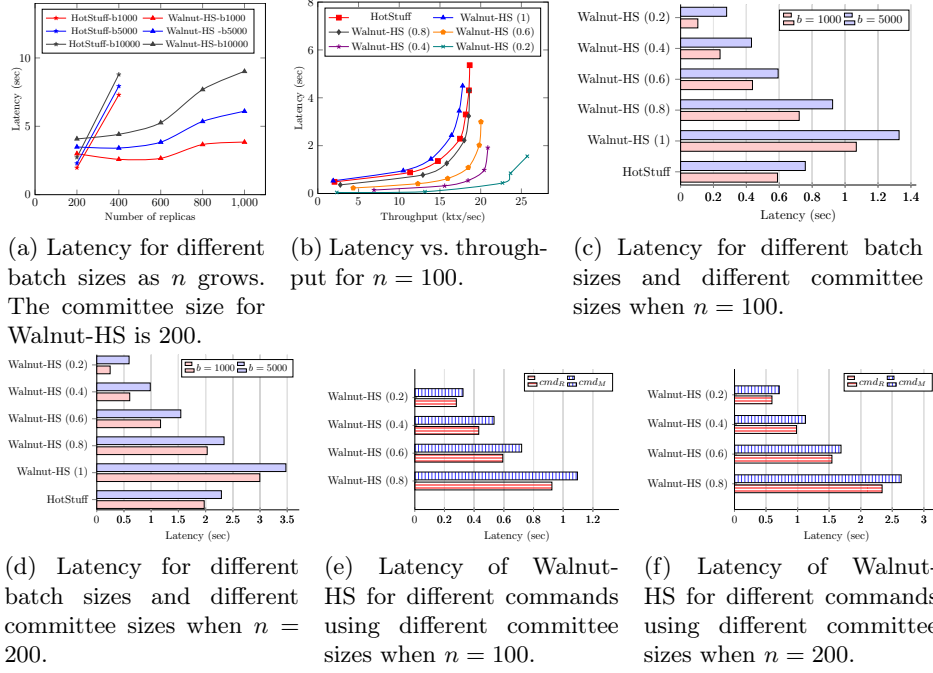


Fig. 4: Performance of the protocols.

it with HotStuff. As shown in Figure 4c-4d, the latency of Walnut-HS (0.6), Walnut-HS (0.4), and Walnut-HS (0.2) is always lower than HotStuff under the same batch size. E.g., when  $n=200$ ,  $b=5,000$ , the latency of Walnut-HS (0.2) is almost only  $\frac{1}{4}$  of that of HotStuff. This result further illustrates that reducing the committee size can indeed effectively enhance the performance of the protocol.

**(4) Latency of different commands.** We evaluate the latency of regular commands and membership commands, and show the results in Figure 4e-4f. The latency of  $cmd_M$  is about 20% higher than  $cmd_R$  for  $n=100$  and 15% for  $n=200$ . This is generally due to the state transfer when the committee rotates. For a large network size, communication overhead is inherently high; hence, the overhead of state transfer decreases relatively. As mentioned in Appendix B.5, the frequency of committee rotation can be set low so that  $cmd_M$  does not appear very often, thus not affecting the overall performance of the protocol.

## 7 Further Discussion and Conclusion

By introducing the first committee-sampling-based scalability enhancement framework for BFT in the partially synchronous network model, this work demonstrates a modular approach to smoothly scale existing BFT protocols to thousands of replicas. We have formally proven the security of our scheme and validated its practicality with an instantiation over HotStuff. In practice, however, choosing the right committee size depends on specific security and fault tolerance

requirements. A large committee may incur high costs and slow execution, while an overly small committee risks compromising fault tolerance and overall security. Thus, the application needs to consider the balance between the committee size and the security of consensus.

**Acknowledgment.** This work is supported by the National Science and Technology Major Project (Grant No. 2022ZD0120304) and Shanghai Science and Technology Innovation Action Plan (Grant No. 24BC3200300, 24BC3200500). Yu Long and Xian Xu are corresponding authors.

## References

1. Bitcoin: A peer-to-peer electronic cash system
2. Abraham, I., Chan, T.H., Dolev, D., Nayak, K., Pass, R., Ren, L., Shi, E.: Communication complexity of byzantine agreement, revisited. In: PODC. pp. 317–326 (2019)
3. Berger, C., Schwarz-Rüsch, S., Vogel, A., Bleeke, K., Jehl, L., Reiser, H.P., Kapitza, R.: Sok: Scalability techniques for bft consensus. In: 2023 IEEE International Conference on Blockchain and Cryptocurrency (ICBC). pp. 1–18. IEEE (2023)
4. Bhangale, A., Liu-Zhang, C., Loss, J., Nayak, K.: Efficient adaptively-secure byzantine agreement for long messages. In: ASIACRYPT. pp. 504–525
5. Buchman, E.: Tendermint: Byzantine fault tolerance in the age of blockchains. Ph.D. thesis, University of Guelph (2016)
6. Camenisch, J., Drijvers, M., Hanke, T., Pignolet, Y.A., Shoup, V., Williams, D.: Internet computer consensus. In: Proceedings of the 2022 ACM Symposium on Principles of Distributed Computing. pp. 81–91 (2022)
7. Castro, M., Liskov, B.: Practical byzantine fault tolerance. In: OSDI. pp. 173–186. USENIX Association (1999)
8. Chan, T.H.H., Pass, R., Shi, E.: Sublinear-round byzantine agreement under corrupt majority. In: PKC. pp. 246–265. Springer (2020)
9. Chernoff, H.: A measure of asymptotic efficiency for tests of a hypothesis based on the sum of observations. The Ann. of Math. Stat. pp. 493–507 (1952)
10. Danezis, G., Kokoris-Kogias, L., Sonnino, A., Spiegelman, A.: Narwhal and tusk: a dag-based mempool and efficient bft consensus. In: EuroSys. pp. 34–50 (2022)
11. Duan, S., Zhang, H.: Foundations of dynamic bft. In: S&P. pp. 1317–1334 (2022)
12. Gao, Y., Lu, Y., Lu, Z., Tang, Q., Xu, J., Zhang, Z.: Dumbo-ng: Fast asynchronous bft consensus with throughput-oblivious latency. In: CCS. pp. 1187–1201 (2022)
13. Gilad, Y., Hemo, R., Micali, S., Vlachos, G., Zeldovich, N.: Algorand: Scaling byzantine agreements for cryptocurrencies. In: SOSP. pp. 51–68 (2017)
14. Gueta, G.G., Abraham, I., Grossman, S., Malkhi, D., Pinkas, B., Reiter, M., Seredinschi, D.A., Tamir, O., Tomescu, A.: Sbft: A scalable and decentralized trust infrastructure. In: DSN. pp. 568–580. IEEE (2019)
15. Guo, B., Lu, Z., Tang, Q., Xu, J., Zhang, Z.: Dumbo: Faster asynchronous bft protocols. In: Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security. pp. 803–818 (2020)
16. Gupta, S., Rahnama, S., Hellings, J., Sadoghi, M.: Resilientdb: Global scale resilient blockchain fabric. Proceedings of the VLDB Endowment **13**(6)
17. Karp, R., Schindelhauer, C., Shenker, S., Vocking, B.: Randomized rumor spreading. In: FOCS. pp. 565–574. IEEE (2000)

18. Kelkar, M., Deb, S., Long, S., Juels, A., Kannan, S.: Themis: Fast, strong order-fairness in byzantine consensus. In: Proceedings of the 2023 acm sigsac conference on computer and communications security. pp. 475–489 (2023)
19. King, V., Saia, J.: Breaking the  $O(n^2)$  bit barrier: scalable byzantine agreement with an adaptive adversary. JACM **58**(4), 1–24 (2011)
20. Kogias, E.K., Jovanovic, P., Gailly, N., Khoffi, I., Gasser, L., Ford, B.: Enhancing bitcoin security and performance with strong consistency via collective signing. In: 25th unix security symposium (usenix security 16). pp. 279–296 (2016)
21. Kogias, E.K., Jovanovic, P., Gailly, N., Khoffi, I., Gasser, L., Ford, B.: Enhancing bitcoin security and performance with strong consistency via collective signing. In: USENIX Security. pp. 279–296 (2016)
22. Kokoris-Kogias, E., Jovanovic, P., Gasser, L., Gailly, N., Syta, E., Ford, B.: Omniledger: A secure, scale-out, decentralized ledger via sharding. In: S&P. pp. 583–598. IEEE (2018)
23. Kwon, J., Buchman, E.: Cosmos whitepaper **27**, 1–32 (2019)
24. LAMPORT, L., SHOSTAK, R., PEASE, M.: The byzantine generals problem. TOPLAS **4**(3), 382–401 (1982)
25. Libra: An introduction to libra pp. 1–12 (2019)
26. Liu, Y., Liu, A., Lu, Y., Pan, Z., Li, Y., Liu, J., Bian, S., Conti, M.: Kronos: A secure and generic sharding blockchain consensus with optimized overhead. Cryptology ePrint Archive, Paper 2024/206 (2024), to appear at NDSS 2025
27. Luu, L., Narayanan, V., Zheng, C., Baweja, K., Gilbert, S., Saxena, P.: A secure sharding protocol for open blockchains. In: CCS. pp. 17–30 (2016)
28. Micali, S., Rabin, M., Vadhan, S.: Verifiable random functions. In: FOCS. pp. 120–130. IEEE (1999)
29. Momose, A., Ren, L.: Constant latency in sleepy consensus. In: Proceedings of the 2022 ACM CCS. pp. 2295–2308 (2022)
30. Mu, K., Yin, B., Asheralieva, A., Wei, X.: Separation is good: A faster order-fairness byzantine consensus. In: 31st Annual Network and Distributed System Security Symposium, NDSS. The Internet Society (2024)
31. Neiheiser, R., Matos, M., Rodrigues, L.: Kauri: Scalable bft consensus with pipelined tree-based dissemination and aggregation. In: Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles. pp. 35–48 (2021)
32. Pass, R., Shi, E.: Thunderella: Blockchains with optimistic instant confirmation. In: EUROCRYPT. pp. 3–33. Springer (2018)
33. Stathakopoulou, C., David, T., Pavlovic, M., Vukolić, M.: [ mir-bft: scalable and robust bft for decentralized networks. Journal of Systems Research **2**(1) (2022)
34. Wang, X., Wang, H., Zhang, H., Duan, S.: Pando: Extremely scalable BFT based on committee sampling. Cryptology ePrint Archive, Paper 2024/664 (2024), <https://eprint.iacr.org/2024/664>.
35. Wood, G., et al.: Ethereum: A secure decentralised generalised transaction ledger. Ethereum project yellow paper **151**(2014), 1–32 (2014)
36. Yin, M., Malkhi, D., Reiter, M.K., Gueta, G.G., Abraham, I.: Hotstuff: Bft consensus with linearity and responsiveness. In: PODC. pp. 347–356 (2019)
37. Zamani, M., Movahedi, M., Raykova, M.: Rapidchain: Scaling blockchain via full sharding. In: CCS. pp. 931–948 (2018)
38. Zhang, Z., Hu, B., Tian, L., Zhou, Y., Liu, J.: Efficient dynamic-committee bft consensus based on hotstuff. Peer-to-Peer Networking and Applications **18**(3), 111 (2025)

## Appendix A Security Proof for Walnut

**Sketch of the security proof.** Figure 5 shows the dependencies between the security goals and the properties. We first prove that  $t_c < \lambda/3$  holds for each committee  $\mathcal{C}_c$  with probability  $1 - \text{negl}(\kappa)$  by applying Chernoff Upper Tail Bound (Lemma 1). Thus, the premises of our five properties are satisfied. Then, we show how to use the five properties to ensure safety and liveness for Walnut.

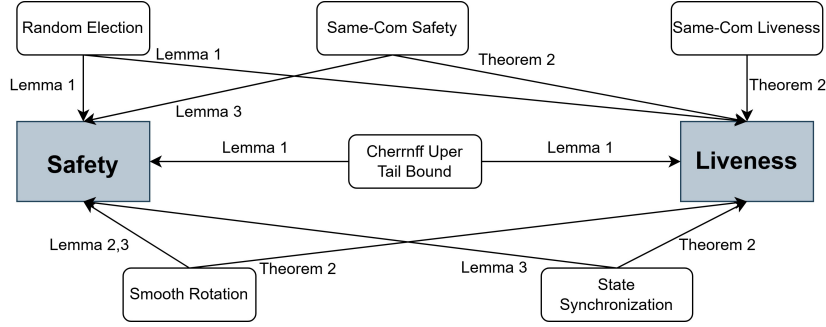


Fig. 5: Dependency of the security of Walnut. The boxes with rounded corners represent properties and theorems, while the gray ones are the security goals to achieve. Arrows indicate the dependencies between these goals and properties.

*Proof of Safety.* When considering safety under the scenario that the committee rotates constantly, a crucial observation is that each committee is responsible for delivering blocks for a certain period. Informally, the key is to show that if a correct replica in  $\mathcal{C}_c$  delivers a block  $b$ , no conflicting block  $b'$  can be delivered.

Next, we show that the conclusion holds for all committee members and non-committee replicas. For correct replicas in  $\mathcal{C}_c$ , the same-com safety property ensures that they will not deliver  $b'$  if  $b'$  conflicts with  $b$  (Lemma 3). For correct replicas in subsequent committees, smooth rotation property ensures: (1) they will only deliver blocks related to the committee they are in (Lemma 2); (2) they will not deliver  $b'$  related to the committee they are in if  $b'$  conflicts with  $b$  (Lemma 3). Thus, all replicas in  $\mathcal{C}_c$  and subsequent committees will not deliver  $b'$  if  $b'$  conflicts with  $b$  (Lemma 3). Additionally, for non-committee replicas, state synchronization ensures they deliver a block if and only if some correct committee members have delivered it. Thus, they will not deliver such a  $b'$  (Lemma 3).

*Proof of Liveness.* Suppose a command  $cmd$  is submitted to all correct replicas in the current committee  $\mathcal{C}_c$ . Liveness of our construction can be considered in two cases: (1) all replicas stay in  $\mathcal{C}_c$  for a sufficiently long time; (2) committee rotation occurs soon.

For case (1), the same-com liveness property ensures that all correct replicas in  $\mathcal{C}_c$  eventually deliver  $cmd$ . Moreover, the state synchronization property additionally ensures that non-committee replicas will also deliver  $cmd$ . Thus, all replicas in the system eventually deliver  $cmd$ .

For case (2), the smooth rotation property ensures correct replicas in subsequent committees continue to propose and deliver new blocks. Thus,  $cmd$  will eventually be proposed. After GST, whenever the committee has no more than  $t$  faulty replicas,  $cmd$  will be delivered by the correct replicas in the committee. Besides, the state synchronization property ensures that non-committee replicas will deliver  $cmd$ .

Next we provide the detailed proof.

**(Chernoff Upper Tail Bound).** Suppose  $\{X_n\}$  is the independent  $\{0, 1\}$ -random variables, and  $X = \sum_i X_i$ . Then for any  $\tau > 0$ :

$$\Pr(X \geq (1 + \tau)E(X)) \leq \exp\left(-\frac{\tau \cdot \min\{\tau, 1\} \cdot E(X)}{3}\right)$$

**Lemma 1** *Let  $\alpha = \frac{1}{3} - \epsilon$  be the fraction of faulty replicas in the system and  $\epsilon$  is a small constant s.t.  $0 < \epsilon \leq \frac{1}{3}$ ,  $\delta$  be the desired failure probability,  $\lambda$  be the number of replicas in the committee. If  $\lambda \geq \frac{3\alpha}{\epsilon^2} \ln \frac{1}{\delta} = O(\kappa)$ , then with probability  $1 - \text{negl}(\kappa)$ , the number of faulty replicas in each committee is less than  $\lambda/3$  and the number of correct replicas in the committee is more than  $2\lambda/3$ .*

*Proof.* We prove this lemma by induction. First, we show that if the number of faulty replicas in  $\mathcal{C}_0$  satisfies  $t_0 < \lambda/3$ , then with probability  $1 - \text{negl}(\kappa)$ , the number of faulty replicas in  $\mathcal{C}_1$  is less than  $\lambda/3$ . Since the committee can only be changed by the delivery of a block  $b$ , which contains a  $cmd_M$ , by Property 1 (Random Election), the election process for the next committee can be modeled as a  $\lambda$ -times independent and repeated experiment like Pando [34]. In each experiment, a determinate replica is chosen randomly to be a committee member among all replicas in the system. This is equivalent to the process in which each replica calls  $\text{VRFGen}()$  to check whether it is a candidate for the committee.

As analyzed in Pando [34],  $\Pr(Y \geq \lambda/3) \leq \delta$  holds with the help of the Chernoff Bound. (see Appendix A.1, Lemma 1, in [34]).

As is argued in [34], in some statistical security parameters (e.g.  $\lambda = \frac{3\alpha}{\epsilon^2} \ln \frac{1}{\delta} = O(\kappa)$ ), the failure probability  $\delta = e^{-\omega(\log \kappa)}$  would be a negligible function. So with probability  $1 - \text{negl}(\kappa)$ , the number of faulty replicas in  $\mathcal{C}_1$  is less than  $\lambda/3$ .

Next we show that if with probability  $1 - \text{negl}(\kappa)$ , the number of faulty replicas in  $\mathcal{C}_c$  is less than  $\lambda/3$ , then with probability  $1 - \text{negl}(\kappa)$ , the number of faulty replicas in  $\mathcal{C}_{c+1}$  is less than  $\lambda/3$ . According to the above analysis, we have

$$\begin{aligned} \Pr(t_{c+1} \geq \lambda/3) &= 1 - \Pr(t_{c+1} \leq \lambda/3) \\ &= 1 - \Pr(t_c \leq \lambda/3) \cdot \Pr(t_{c+1} \leq \lambda/3 | t_c \leq \lambda/3) \\ &\quad - \Pr(t_c \geq \lambda/3) \cdot \Pr(t_{c+1} \leq \lambda/3 | t_c \geq \lambda/3) \\ &\leq 1 - \Pr(t_c \leq \lambda/3) \cdot \Pr(t_{c+1} \leq \lambda/3 | t_c \leq \lambda/3) \\ &= 1 - (1 - \text{negl}(\kappa)) \cdot (1 - \delta) \\ &= \delta + \text{negl}(\kappa) - \delta \cdot \text{negl}(\kappa) \\ &= \text{negl}(\kappa) \end{aligned}$$

Since  $t_1 < \lambda/3$  with probability  $1 - \text{negl}(\kappa)$ , the number of faulty replicas in  $\mathcal{C}_2$  is less than  $\lambda/3$  with probability  $1 - \text{negl}(\kappa)$ . By induction, with probability  $1 - \text{negl}(\kappa)$ , the number of faulty replicas in each subsequent committee is less than  $\lambda/3$ .

**Lemma 2** *If  $t_c < \lambda/3$  holds for the committee  $\mathcal{C}_c$ , then a correct replica  $\mathcal{P}_i$  in  $\mathcal{C}_c$  can only deliver a block  $b$  related to the  $c$ -th committee.*

*Proof.* If  $\mathcal{C}_c$  is the latest committee, we only need to prove that  $\mathcal{P}_i$  cannot deliver any block related to the previous committees. If  $\mathcal{C}_c$  is the first committee, this lemma naturally holds. If not,  $\mathcal{P}_i$  must have delivered a block  $b'$  containing a  $\text{cmd}_M$  related to  $\mathcal{C}_{c-1}$  before it enters  $\mathcal{C}_c$ . Denote the first block  $\mathcal{P}_i$  delivers related to  $\mathcal{C}_c$  by  $b''$ . If  $\mathcal{P}_i$  delivers a block  $b'''$  related to  $\mathcal{C}_{c-1}$  after it enters  $\mathcal{C}_c$ , then  $b'''$  must be between  $b'$  and  $b''$ . However, by Property 4 (Smooth Rotation),  $b''$  must directly extend  $b'$ . Therefore, such a  $b'''$  does not exist. The lemma thus holds.

If  $\mathcal{C}_c$  is not the latest committee, similar to the above proof,  $\mathcal{P}_i$  cannot deliver any block related to the previous committees. We need to additionally prove that  $\mathcal{P}_i$  cannot deliver any block related to the subsequent committees.  $\mathcal{P}_i$  updates its committee only when it delivers a block  $b'$  containing a  $\text{cmd}_M$  related to  $\mathcal{C}_c$ . However, at that time, the new committee  $\mathcal{C}_{c+1}$  will take over the work of  $\mathcal{C}_c$ .  $\mathcal{P}_i$  will deliver new blocks as a replica in  $\mathcal{C}_{c+1}$  or by state synchronization with some replicas in  $\mathcal{C}_{c+1}$ , but no longer as a replica in  $\mathcal{C}_c$ .

**Lemma 3** *If  $t_c < \lambda/3$  holds for the committee  $\mathcal{C}_c$  and a correct replica  $\mathcal{P}_i$  delivers a block  $b$  related to  $\mathcal{C}_c$  in view  $v$ , then all correct replicas in the system eventually deliver  $b$  in view  $v$ .*

*Proof.* Since  $\mathcal{P}_i$  delivers  $b$  in view  $v$ , by Property 5 (State Synchronization) (ii), at least one correct replica  $\mathcal{P}_j$  in  $\mathcal{C}_c$  has delivered  $b$  in view  $v$ . There are two cases for any correct replica  $\mathcal{P}_k$  in the system: (1)  $\mathcal{P}_k$  is in  $\mathcal{C}_c$ ; (2)  $\mathcal{P}_k$  is not in  $\mathcal{C}_c$ .

Case (1): If  $\mathcal{C}_c$  is the latest committee, let all the blocks  $\mathcal{P}_j$  delivers until  $b$  which are related to the  $c$ -th committee be  $[b_1, b_2, \dots, b_s, b]$ . By Property 4,  $\mathcal{P}_k$  eventually delivers  $b_1, b_2, \dots, b_s, b$  and these are all the blocks  $\mathcal{P}_k$  delivers until  $b$  that are related to the  $c$ -th committee. Moreover, by Property 2 (Same-Com Safety), the sequence  $\mathcal{P}_k$  delivers these blocks is exactly the same as that of  $\mathcal{P}_j$ . Since  $\mathcal{P}_j$  delivers  $b$  in view  $v$ , then  $\mathcal{P}_k$  also delivers  $b$  in view  $v$ .

If  $\mathcal{C}_c$  is not the latest committee, let all the blocks  $\mathcal{P}_j$  delivers which are related to the  $c$ -th committee be  $[b_1, b_2, \dots, b, \dots, b_m]$ . By Property 4, we have: (1)  $\mathcal{P}_k$  eventually delivers  $b_1, b_2, \dots, b, \dots, b_m$  and these are all the blocks  $\mathcal{P}_k$  delivers which are related to the  $c$ -th committee; (2) only  $b_m$  contains a  $\text{cmd}_M$ . Let  $\mathcal{P}_l$  be the first correct replica in  $\mathcal{C}_c$  that delivers  $b_m$ . By Property 4,  $\mathcal{P}_l$  must have delivered  $[b_1, b_2, \dots, b, \dots, b_{m-1}]$  before delivering  $b_m$  and these are all the blocks  $\mathcal{P}_l$  delivers which are related to  $\mathcal{C}_c$ . Before  $\mathcal{P}_l$  delivers  $b_m$ ,  $\mathcal{C}_c$  is still the latest committee. Thus, by Property 2, the sequence  $\mathcal{P}_k$  delivers these blocks is exactly the same as that of  $\mathcal{P}_j$  and  $\mathcal{P}_l$ . Moreover, by Lemma 2, the subsequent committees cannot deliver any new block related to  $\mathcal{C}_c$ . In other words,  $[b_1, b_2, \dots, b, \dots, b_m]$

are all the blocks related to  $\mathcal{C}_c$  that will be delivered. Since  $\mathcal{P}_j$  delivers  $b$  in view  $v$ , then  $\mathcal{P}_k$  also delivers  $b$  in view  $v$ .

Case (2): Since  $\mathcal{P}_j$  has delivered  $b$  in view  $v$  and  $\mathcal{P}_k$  is not in  $\mathcal{C}_c$ , by Property 5 (i),  $\mathcal{P}_k$  eventually delivers  $b$  in  $v$ .

**Theorem 1** (Safety). *If a correct replica delivers a command  $cmd$  before delivering  $cmd'$ , then no correct replica delivers a command  $cmd'$  without first delivering  $cmd$  with probability  $1 - \text{negl}(\kappa)$ .*

*Proof.* Let  $b$  and  $b'$  be the blocks containing  $cmd$  and  $cmd'$ , respectively. As the order of commands is fixed within a block, we only need to prove that if a correct replica delivers a block  $b$  before delivering  $b'$ , then no correct replica delivers  $b'$  without first delivering  $b$  with probability  $1 - \text{negl}(\kappa)$ .

Towards a contradiction, we assume a correct replica  $\mathcal{P}_i$  delivers  $b$  in view  $v_1$  before delivering  $b'$  in view  $v'_1$  where  $v_1 < v'_1$  and another correct replica  $\mathcal{P}_j$  delivers  $b'$  in view  $v'_2$  before delivering  $b$  in view  $v_2$  where  $v'_2 < v_2$ .

Let the current committee be  $\mathcal{C}_c$ , there are two cases for  $b$  and  $b'$ : (1)  $b$  and  $b'$  are related to the same committee  $\mathcal{C}_k$  where  $k \leq c$ ; (2)  $b$  and  $b'$  are related to different committees  $\mathcal{C}_k$  and  $\mathcal{C}_{k'}$  respectively. W.l.o.g., we assume  $k < k' \leq c$ .

Case (1): By Lemma 1,  $t_k < \lambda/3$  holds for the  $k$ -th committee  $\mathcal{C}_k$  with probability  $1 - \text{negl}(\kappa)$ . When  $t_k < \lambda/3$ , by Lemma 3, we know that  $v_2 = v_1$  and  $v'_2 = v'_1$ . Thus  $v'_2 > v_2$ , a contradiction with  $v'_2 < v_2$ .

Case (2): By Lemma 1,  $t_k < \lambda/3$  holds for the  $k$ -th committee  $\mathcal{C}_k$  with probability  $1 - \text{negl}(\kappa)$ . When  $t_k < \lambda/3$ , by Lemma 3, we know that  $v_2 = v_1$  and  $\mathcal{P}_j$  has delivered all the blocks related to  $\mathcal{C}_k$  with the same sequence as  $\mathcal{P}_i$ . Assume all the blocks related to  $\mathcal{C}_k$  are delivered from view  $v_s$  to view  $v_e$ . By Lemma 2, since  $b'$  is related to  $\mathcal{C}_{k'}$  after  $\mathcal{C}_k$ ,  $\mathcal{P}_j$  cannot deliver  $b'$  until it updates the committee to  $\mathcal{C}_{k'}$  by the delivery of a block containing a  $cmd_M$  in a view  $v''$  where  $v'' \geq v_e$ , i.e.,  $v'_2 > v_e$ . Since  $v_s \leq v_1 = v_2 \leq v_e$ , we have  $v'_2 > v_2$ , a contradiction with  $v'_2 < v_2$ .

**Theorem 2** (Liveness). *If a command  $cmd$  is submitted to all correct replicas, then all correct replicas eventually deliver  $cmd$  with probability  $1 - \text{negl}(\kappa)$ .*

*Proof.* Let the current committee be  $\mathcal{C}_c$ . If the committee does not change, since  $cmd$  is submitted to all correct replicas, all correct replicas in  $\mathcal{C}_c$  have received  $cmd$ . By Lemma 1,  $t_c < \lambda/3$  holds for  $\mathcal{C}_c$  with probability  $1 - \text{negl}(\kappa)$ . When  $t_c < \lambda/3$ , by Property 3, all correct replicas in  $\mathcal{C}_c$  eventually deliver  $cmd$ . Moreover, by Property 5 (i), all correct replicas that are not in  $\mathcal{C}_c$  will also deliver  $cmd$ . Thus, all correct replicas will eventually deliver  $cmd$ .

If the committee changes before some correct replicas in  $\mathcal{C}_c$  deliver  $cmd$ , since  $cmd$  is submitted to all correct replicas, then all correct replicas in each subsequent committee have received  $cmd$ . By Property 4, correct replicas in each subsequent committee continue to propose and deliver new blocks. Thus, eventually in a committee  $\mathcal{C}_{c'}$  where  $c' > c$ , a correct replica will propose a block containing  $cmd$ . By Lemma 1,  $t_{c'} < \lambda/3$  holds for  $\mathcal{C}_{c'}$  with probability  $1 - \text{negl}(\kappa)$ . Thus, after GST, the probability that  $cmd$  is not delivered is therefore bounded by  $\text{negl}(\kappa)$ . By Lemma 3, all correct replicas in the system eventually deliver  $cmd$ .



## Appendix B Details of the Walnut-HS protocol

Algorithm 1 shows the utility functions, Algorithm 2, 3, and 4 present the generic workflow of the committee election,  $M\text{-}\Pi_{\text{HS}}$ , and the state transfer module respectively. All the modifications against the original  $\Pi_{\text{HS}}$  are marked in purple in Algorithm 1 and Algorithm 3. In the following subsections, we give a detailed explanation of each algorithm and its corresponding module.

### B.1 Data Structure

**Block.** A block  $b$  has the form  $\langle v, c, h, \text{batch} \rangle$ , where  $v$  is a view number,  $c$  is a committee number,  $h$  is a hash value of the last block, and  $\text{batch}$  contains some commands, including  $\text{cmd}_R$  and  $\text{cmd}_M$ . Compared to the original basic HotStuff,  $c$  is an additional parameter. If a block  $b$  is proposed by a correct replica in  $\mathcal{C}_c$ , then we say that  $b$  is related to  $\mathcal{C}_c$ . Moreover, if  $b$  contains a command  $\text{cmd}$ , we say that  $\text{cmd}$  is related to the  $c$ -th committee. For ease of expression, we use  $b_c^v$  to denote a block related to  $\mathcal{C}_c$  and delivered in view  $v$ . Unless otherwise stated, when we say that a correct replica in  $\mathcal{C}_c$  delivers a block  $b$  or a command  $\text{cmd}$ , then  $b$  or  $\text{cmd}$  is restricted to be related to the  $c$ -th committee.

**Quorum certificate (QC).** A quorum certificate  $QC$  has the form  $\langle \text{TYPE}, v, c, h, \mathcal{S}_c \rangle$ , where  $v$  is a view number,  $c$  is a committee number,  $h$  is a hash value, and  $\mathcal{S}_c$  includes  $\lambda - t$  signatures from  $\mathcal{C}_c$  on  $h$ . We use  $QC_c^v$  to denote a QC that is related to  $\mathcal{C}_c$  and is formed in view  $v$ . Similar to prior work, we say that a  $QC_c^v$  directly refers to a block  $b_c^v$  if  $h = \mathcal{H}(b_c^v)$  (denoted by " $b_c^v \Leftarrow QC_c^v$ "). When a committee changes by the delivery of a block  $b_c^{v'}$  led by  $b_c^v$  ( $v > v'$ ), we say that  $QC_c^v$  refers to  $b_c^{v'}$  (denoted by " $b_c^{v'} \Leftarrow^* QC_c^v$ "). A certificate  $QC_c^v$  is valid if all signatures in  $\mathcal{S}_c$  are valid and produced by committee members from  $\mathcal{C}_c$ . This requirement is used to deal with the rotation of committees.

**Branches.** The *branch* led by a block  $b$  is the path from  $b$  to the genesis block. If a block  $b'$  is in the *branch* led by  $b$  and  $b' \neq b$ , we say  $b'$  is an ancestor of  $b$  and  $b$  extends  $b'$ . For ease of expression, we sometimes abuse the notation to denote the relation between a quorum certificate  $qc$  and a block. For example, when we say  $qc$  extends  $b$ , it means that the block  $qc$  refers to extends  $b$ . Two blocks, certificates, or branches are conflicting if neither one extends the other.

### B.2 The Committee Election Process

The committee election process generates  $\text{cmd}_M$  for the committee rotation, as shown in Algorithm 2. Each replica holds a difficulty value  $D$  s.t.  $\frac{D}{E} = \frac{\lambda}{n}$ , where  $E$  is the space size of the random number  $\rho$ . Thus, the expected number of replicas holding a valid random number in each view is  $\lambda$ . In each view  $v_{ce}$ , every replica  $\mathcal{P}_i$  waits for  $\langle \text{NEW-VRF-VIEW}, v' \rangle$  messages from the current committee s.t.  $v' \geq v_{ce}$  (line 1). After  $\mathcal{P}_i$  receives  $\lambda - t$  such messages,  $\mathcal{P}_i$  updates  $v_{ce}$  to  $v' + 1$  (line 2). Then  $\mathcal{P}_i$  calls  $\text{VRFGen}()$  to generate a random number  $\rho$  and a corresponding proof  $\pi$  (line 3). If  $\rho \leq D$ , then  $\mathcal{P}_i$  is a candidate for the next committee, and  $\mathcal{P}_i$  sends a VRF message to the current committee (lines 4-5). All replicas getting the public key of  $\mathcal{P}_i$  can verify the validity of the message.

**Algorithm 1** Utilities

---

```

1: function VrfyProp( $highQC, b, c''', nextcom$ )
2:   parse  $highQC$  as  $\langle \text{PREPARE}, v', c', h', \mathcal{S}_{c'} \rangle$ 
3:   parse  $b$  as  $\langle v, c, h, batch \rangle$ 
4:   denote the  $cmd_M$  set in  $batch$  as  $nextcom'$ 
5:   if  $c \neq c'''$  then ▷ validity of committee
6:     return 0
7:   if  $\text{VrfySig}(\text{PREPARE} || v' || c' || h', \mathcal{S}_{c'}) \neq 1$  then
8:     return 0
9:    $nextcom'' \leftarrow$  the top  $\lambda$  messages with the smallest  $\rho$  from  $nextcom$ 
10:  if  $nextcom' \neq \emptyset$  then ▷ validity of  $nextcom$ 
11:    if  $nextcom' \neq nextcom''$  or any previous block related to  $\mathcal{C}_c$  led by  $b$  contains  $cmd_M$  then
12:      return 0
13:     $b' \stackrel{*}{\leftarrow} highQC$ 
14:    parse  $b'$  as  $\langle v''', c', h''', batch''' \rangle$ 
15:    if  $h = h' \wedge c = c'$  then ▷ validity of proposal
16:       $b'' \leftarrow lockedQC$ 
17:      parse  $b''$  as  $\langle v'', c'', h'', batch'' \rangle$ 
18:      if  $b$  extends from  $b''$  then return 1 ▷ safety rule
19:      if  $v' > v''$  then return 1 ▷ liveness rule
20:    if  $h = \mathcal{H}(b') \wedge c = c' + 1 \wedge b'$  contains  $cmd_M$  then
21:       $b'''' \stackrel{*}{\leftarrow} lockedQC$ 
22:      parse  $b''''$  as  $\langle v'''', c''', h''', batch'''' \rangle$ 
23:      if  $b$  extends from  $b''''$  then return 1 ▷ safety rule
24:      if  $v' > v''''$  then return 1 ▷ liveness rule
25:    return 0
26: function GenQC( $\mathcal{M}_{\text{type}}$ )
27:   parse  $\mathcal{M}_{\text{type}}$  as  $\{\langle \text{TYPE}, v, c, h, \sigma_i \rangle\}_{i \in I}$  ▷  $I$  is the index set
28:    $\mathcal{S}_c \leftarrow \{\sigma_i\}_{i \in I}$ 
29:   return  $QC \leftarrow \langle \text{TYPE}, v, c, h, \mathcal{S}_c \rangle$ 
30: function Deliver( $v, c, batch$ )
31:   for each  $cmd_R \in batch$  do
32:      $re \leftarrow$  execute  $cmd_R$ 
33:     send  $\langle \text{REPLY}, v, re \rangle$  to the client
34:   denote the  $cmd_M$  set in  $batch$  as  $nextcom'$ 
35:   if  $|nextcom'| = \lambda$  then ▷ update committee
36:      $c \leftarrow c + 1$ 
37:      $\mathcal{C} \leftarrow \perp$ 
38:     for each  $\langle j, \rho, \pi \rangle \in nextcom'$  do
39:        $\mathcal{C} \leftarrow \mathcal{C} \cup j$ 
40:   return  $c, \mathcal{C}$ 

```

---

**B.3 The  $M\text{-}\Pi_{\text{HS}}$  Process**

$M\text{-}\Pi_{\text{HS}}$  is a modified version of  $\Pi_{\text{HS}}$ . While the "prepare", "lock", and "commit" rules for blocks in a single committee are the same, the following two main modifications are introduced to get in accordance with Walnut,

**Algorithm 2** Committee Election

---

**Init:**  $\mathcal{P}_i$  initializes the committee election view number  $v_{ce}$  as 0, a difficulty value  $D$  as  $E \cdot \frac{\lambda}{n}$ . The committee number  $c$  and committee  $\mathcal{C}$  are the same as in Algorithm 3.

---

```

1: upon receiving  $\lambda - t \langle \text{NEW-VRF-VIEW}, v' \rangle$  from  $\mathcal{C}$  s.t.  $v' \geq v_{ce}$  do
2:    $v_{ce} \leftarrow v' + 1$ 
3:    $(\rho, \pi) \leftarrow \text{VRFGen}_{sk_i}(v_{ce} || c)$ 
4:   if  $\rho \leq D$  then
5:     send  $\langle \text{VRF}, v_{ce}, c, \rho, \pi \rangle$  to  $\mathcal{C}$ 

```

---

- The process for membership commands.
- An efficient catch-up mechanism to address the possible network asynchrony during committee rotations.

Concretely, as shown in Algorithm 3,  $\text{M-}\Pi_{\text{HS}}$  has four phases as  $\Pi_{\text{HS}}$  and an additional catch-up mechanism. All modifications are marked in purple.

**PREPARE phase.** In each view, every committee member  $\mathcal{P}_i$  collects VRF messages from all replicas (line 1). Then  $\mathcal{P}_i$  calls  $\text{VRFVrfy}()$  to check whether the random number is valid. If so,  $\mathcal{P}_i$  adds this random number with the corresponding proof and its sender to the *nextcom* set (lines 2-3).

After receiving  $\lambda - t \langle \text{NEW-VIEW}, v-1, c, * \rangle$  messages from the committee, the leader  $\mathcal{P}_\ell$  selects *highQC* with the highest view number (line 5). Then  $\mathcal{P}_\ell$  packages *cmd<sub>R</sub>* it receives into *batch* (line 8). If  $\mathcal{P}_\ell$  has collected enough *cmd<sub>M</sub>* and no previous block proposed by this committee contains *cmd<sub>M</sub>*,  $\mathcal{P}_\ell$  adds the top  $\lambda$  elements with the smallest  $\rho$  from *nextcom* to *batch* (lines 9-11). If the committee remains the same as the committee where *highQC* is formed, then  $\mathcal{P}_\ell$  generates a block  $\langle v, c, h', \text{batch} \rangle$  (lines 12-13), which is the same as that in  $\Pi_{\text{HS}}$ . Otherwise, the block would be  $\langle v, c, \mathcal{H}(b'), \text{batch} \rangle$  instead, where the parent is the block *highQC* refers to (lines 14-15). Then  $\mathcal{P}_\ell$  sends a  $\langle \text{PREPARE}, v, c, \text{highQC}, b \rangle$  message to the committee (line 16) and sends the block to all replicas that are not in the committee (line 17).

After receiving a  $\langle \text{PREPARE}, v, c, \text{highQC}, b \rangle$  message from the leader  $\mathcal{P}_\ell$ , every replica  $\mathcal{P}_i$  calls  $\text{VrfyProp}()$  (in Algorithm 1) to verify the block (line 28).  $\mathcal{P}_i$  first verifies the committee number of the block (Algorithm 1, lines 5-6). Then  $\mathcal{P}_i$  checks whether all signatures in *highQC* are valid (Algorithm 1, lines 7-8). If the block  $b$  contains *cmd<sub>M</sub>*,  $\mathcal{P}_i$  additionally verifies whether: (1) no previous block contains *cmd<sub>M</sub>*; (2) the *cmd<sub>M</sub>* set is consistent with its own (Algorithm 1, lines 9-12). If both conditions are satisfied, then the *cmd<sub>M</sub>* is valid. Next, we need to consider two cases. If the committee remains the same as the committee where *highQC* is formed (Algorithm 1, line 15), then our verification is exactly the same as  $\Pi_{\text{HS}}$ . In other words,  $\mathcal{P}_i$  needs to check whether  $b$  extends the block *lockedQC* directly refers to (Algorithm 1, line 18, safety rule) or the view of *highQC* is greater than that of *lockedQC* (Algorithm 1, line 19, liveness rule). The block is valid if any one of the conditions is satisfied. However, if the committee rotation happens (i.e.,  $b$  is the first block proposed in the current committee  $\mathcal{C}_c$ ), more verifications are needed. Let  $b'$  be the block *highQC* refers to (i.e.,  $b' \stackrel{*}{\leftarrow} \text{highQC}$ ),

$b''''$  be the block *lockedQC* refers to (i.e.,  $b'''' \stackrel{*}{\leftarrow} \text{lockedQC}$ ).  $\mathcal{P}_i$  first verifies whether: (1)  $b$  directly extends  $b'$ ; (2) *highQC* is formed in the last committee; (3)  $b'$  contains  $\text{cmd}_M$  thus  $b'$  is proposed by the last committee (Algorithm 1, line 20). If all the conditions are satisfied, then  $\mathcal{P}_i$  uses  $b''''$  for safety check (Algorithm 1, line 23) and liveness check (Algorithm 1, line 24). If the proposal passes all the checks, then  $\mathcal{P}_i$  signs the block and sends its vote to the leader (lines 16-17).

**PRE-COMMIT phase.** After receiving  $\lambda-t$  V-PREPARE messages from the committee,  $\mathcal{P}_\ell$  use the votes to generate *prepareQC* and send a PRE-COMMIT message containing *prepareQC* to the committee (lines 19-20).

When receiving a PRE-COMMIT message from the leader, each replica  $\mathcal{P}_i$  verifies the signatures in the *prepareQC'* (line 33). If the QC is valid,  $\mathcal{P}_i$  updates its local *prepareQC* and sends a pre-commit vote message to  $\mathcal{P}_\ell$  (lines 34-36).

**COMMIT phase.** The workflow of the commit phase is very similar to the pre-commit phase, and we omit the details.

**DECIDE phase.** The workflow for  $\mathcal{P}_\ell$  is very similar to the pre-commit phase where  $\mathcal{P}_\ell$  collects votes, generates *commitQC*, and sends it to the committee (lines 24-26).

Upon receiving a  $\langle \text{DECIDE}, v, c, \text{commitQC}', \perp \rangle$  message from the leader, where all signatures in *commitQC'* are valid,  $\mathcal{P}_i$  updates its local *commitQC* and prepares to deliver the block (lines 43-46). If  $b$  is the first block containing  $\text{cmd}_M$  proposed by the committee,  $\mathcal{P}_i$  needs to record *commitQC* in its *comchange* set (lines 48-49). This is because  $\mathcal{P}_i$ , as a committee member in  $\mathcal{C}_c$ , is responsible for the recovery of the blocks proposed by  $\mathcal{C}_c$  and *commitQC* serves as a proof for the delivery of these blocks.  $\mathcal{P}_i$  then delivers all the new blocks led by  $b$  (lines 50-63). For all regular commands in a block,  $\mathcal{P}_i$  execute them and send the results to the client (Algorithm 1, lines 31-33). For membership commands,  $\mathcal{P}_i$  updates the committee by the *nextcom'* contained in the block (Algorithm 1, lines 34-39). If the committee remains the same after the delivery of a block,  $\mathcal{P}_i$  simply sends the hash value of the block delivered to non-committee replicas for state transfer (line 63). However, if committee rotation occurs in the delivery of  $b'$  (line 53), the delivery process is interrupted, and all blocks after  $b'$  will not be delivered. This ensures that only the blocks proposed by the next committee will be recognized when the committee rotates.  $\mathcal{P}_i$  then records the *gap* blocks in *catch* (lines 55-56). Such blocks may exist because  $b'$  may not be the block *commitQC'* directly refers to. However, we need to record these blocks to verify the validity of *commitQC'*. For state transfer,  $\mathcal{P}_i$  then sends all the QCs it records locally to the next committee and sends the hash value of the block delivered to replicas that are not in the current committee or the next committee (lines 57-58). If  $\mathcal{P}_i$  is in the next committee, it waits for ST messages from the current committee to ensure that its QCs are the latest (lines 59-60). If  $\mathcal{P}_i$  has done all the work or has experienced a timeout, then  $\mathcal{P}_i$  sends a NEW-VRF-VIEW message to all replicas as well as a NEW-VIEW message to the next leader and then enters the next view (lines 64-70).

**CATCH-UP mechanism.** Finally, the catch-up mechanism is to ensure that if a replica misses some blocks, it can recover them even when committee rotation occurs. Upon repeated timeout, each replica  $\mathcal{P}_i$  sends a  $\langle \text{CATCH-UP}, v', c \rangle$  message to the current committee, where  $v'$  is the view number of its highest block (lines 71-73).

After receiving a CATCH-UP message from  $\mathcal{P}_j$ , if  $\mathcal{P}_i$  is in the corresponding committee and the view of its highest block is higher than  $v'$ ,  $\mathcal{P}_i$  sends a  $\langle \text{HIST}, v', c', hist, catch', qc \rangle$  message to  $\mathcal{P}_j$ , where  $hist$  and  $catch'$  contain all the blocks related to  $\mathcal{C}_{c'}$  and are higher than view  $v' + 1$ , while  $qc$  is the highest  $commitQC$  formed in  $\mathcal{C}_{c'}$  which  $\mathcal{P}_i$  records (lines 74-84).

Upon receiving a HIST message from  $\mathcal{P}_j$ ,  $\mathcal{P}_i$  checks whether: (1) the first block in  $hist$  directly extends the highest block it records; (2) all blocks in  $hist$  and  $catch'$  are related to  $\mathcal{C}_{c'}$  and form a chain (line 87). If so,  $\mathcal{P}_i$  then verifies the validity of  $qc$  and delivers all the new blocks as in the decide phase (lines 91-92). Finally,  $\mathcal{P}_i$  updates its view number and enters the new view (line 93).

---

**Algorithm 3** M- $\Pi_{\text{HS}}$ 


---

**Init:**  $\mathcal{P}_i$  initializes the view number  $v$  as 0, committee number  $c$  as 0, committee  $\mathcal{C}$  as  $\mathcal{C}_0$ , a block set  $catch$  as  $\emptyset$ , a committee change proof set  $comchange$  as  $\emptyset$ ;

---

**Init:** In each view,  $\mathcal{P}_i$  initializes four message sets  $\mathcal{M}_{\text{prep}}$ ,  $\mathcal{M}_{\text{preCom}}$ ,  $\mathcal{M}_{\text{com}}$ , and  $\mathcal{M}_{\text{catch}}$  as  $\emptyset$ , a command set  $batch$  as  $\emptyset$ , a VRF output set  $nextcom$  as  $\emptyset$ ;

**Init:**  $\mathcal{P}_i$  starts a timer  $\Delta$  when entering a new view

- 1: **upon receiving**  $\langle \text{VRF}, v, c, \rho, \pi \rangle$  from  $\mathcal{P}_j$  **do**
- 2:   **if**  $\text{VRFVrfy}_{pk_j}(\rho, \pi, v || c) = 1$  **then**
- 3:      $nextcom \leftarrow nextcom \cup \langle j, \rho, \pi \rangle$
- As a leader**  $\mathcal{P}_i$ :
- 4: **upon receiving**  $\lambda - t$   $\langle \text{NEW-VIEW}, v - 1, c, * \rangle$  from  $\mathcal{C}$  **do**  $\triangleright$  PREPARE
- 5:   sort  $highQC$  that has the highest view number
- 6:   parse  $highQC$  as  $\langle \text{PREPARE}, v', c', h', \mathcal{S}_{c'} \rangle$
- 7:    $b' \xleftarrow{*} highQC$
- 8:    $batch \leftarrow \{cmd_R^1, \dots, cmd_R^B\}$   $\triangleright B$  is the batch size
- 9:   **if**  $|nextcom| \geq \lambda$  and all blocks related to  $\mathcal{C}_c$  led by  $b'$  contain no  $cmd_M$  **then**
- 10:      $nextcom \leftarrow$  the top  $\lambda$  elements with the smallest  $\rho$  from  $nextcom$
- 11:      $batch \leftarrow batch \cup nextcom$
- 12:   **if**  $c' = c$  **then**
- 13:      $b \leftarrow \langle v, c, h', batch \rangle$
- 14:   **else**
- 15:      $b \leftarrow \langle v, c, \mathcal{H}(b'), batch \rangle$
- 16:   send  $\langle \text{PREPARE}, v, c, highQC, b \rangle$  to  $\mathcal{C}$
- 17:   send  $\langle \text{BLOCK}, v, c, b \rangle$  to all replicas but  $\mathcal{C}$
- 18: **upon receiving**  $\lambda - t$  valid v-PREPARE messages from  $\mathcal{C}$  **do**  $\triangleright$  PRE-COMMIT
- 19:    $prepareQC \leftarrow \text{GenQC}(\mathcal{M}_{\text{prep}})$   $\triangleright \mathcal{M}_{\text{prep}}$  is the message set
- 20:   send  $\langle \text{PRE-COMMIT}, v, c, prepareQC, \perp \rangle$  to  $\mathcal{C}$
- 21: **upon receiving**  $\lambda - t$  valid v-PRE-COM messages from  $\mathcal{C}$  **do**  $\triangleright$  COMMIT
- 22:    $lockedQC \leftarrow \text{GenQC}(\mathcal{M}_{\text{preCom}})$   $\triangleright \mathcal{M}_{\text{preCom}}$  is the message set
- 23:   send  $\langle \text{COMMIT}, v, c, lockedQC, \perp \rangle$  to  $\mathcal{C}$
- 24: **upon receiving**  $\lambda - t$  valid v-COM messages from  $\mathcal{C}$  **do**  $\triangleright$  DECIDE

25:  $commitQC \leftarrow \text{GenQC}(\mathcal{M}_{\text{com}})$   $\triangleright \mathcal{M}_{\text{com}}$  is the message set  
 26: send  $\langle \text{DECIDE}, v, c, commitQC, \perp \rangle$  to  $\mathcal{C}$   
 As a replica  $\mathcal{P}_i$ :  
 27: **upon** receiving  $\langle \text{PREPARE}, v, c, highQC, b \rangle$  from  $\mathcal{P}_\ell$  **do**  $\triangleright$  PREPARE  
 28:   **if**  $\text{VrfyProp}(highQC, b, c, nextcom) = 1$  **then**  
 29:      $\sigma_{\text{prep}, i} \leftarrow \text{Sign}_i(\text{PREPARE} || v || c || \mathcal{H}(b))$   
 30:     send  $\langle \text{V-PREPARE}, v, c, \mathcal{H}(b), \sigma_{\text{prep}, i} \rangle$  to  $\mathcal{P}_\ell$   
 31: **upon** receiving  $\langle \text{PRE-COMMIT}, v, c, prepareQC', \perp \rangle$  from  $\mathcal{P}_\ell$  **do**  $\triangleright$  PRE-COMMIT  
 32:   parse  $prepareQC'$  as  $\langle \text{PREPARE}, v, c, h, \mathcal{S}_c \rangle$   
 33:   **if**  $\text{VrfySig}(\text{PREPARE} || v || c || h, \mathcal{S}_c) = 1$  **then**  
 34:      $prepareQC \leftarrow prepareQC'$   
 35:      $\sigma_{\text{preCom}, i} \leftarrow \text{Sign}_i(\text{PRE-COMMIT} || v || c || h)$   
 36:     send  $\langle \text{V-PRE-COM}, v, c, h, \sigma_{\text{preCom}, i} \rangle$  to  $\mathcal{P}_\ell$   
 37: **upon** receiving  $\langle \text{COMMIT}, v, c, lockedQC', \perp \rangle$  from  $\mathcal{P}_\ell$  **do**  $\triangleright$  COMMIT  
 38:   parse  $lockedQC'$  as  $\langle \text{PRE-COMMIT}, v, c, h, \mathcal{S}_c \rangle$   
 39:   **if**  $\text{VrfySig}(\text{PRE-COMMIT} || v || c || h, \mathcal{S}_c) = 1$  **then**  
 40:      $lockedQC \leftarrow lockedQC'$   
 41:      $\sigma_{\text{com}, i} \leftarrow \text{Sign}_i(\text{COMMIT} || v || c || h)$   
 42:     send  $\langle \text{V-COM}, v, c, h, \sigma_{\text{com}, i} \rangle$  to  $\mathcal{P}_\ell$   
 43: **upon** receiving  $\langle \text{DECIDE}, v, c, commitQC', \perp \rangle$  from  $\mathcal{P}_\ell$  **do**  $\triangleright$  DECIDE  
 44:   parse  $commitQC'$  as  $\langle \text{COMMIT}, v, c, h, \mathcal{S}_c \rangle$   
 45:   **if**  $\text{VrfySig}(\text{COMMIT} || v || c || h, \mathcal{S}_c) = 1$  **then**  
 46:      $commitQC \leftarrow commitQC'$   
 47:      $b \stackrel{*}{\leftarrow} commitQC'$   
 48:     **if**  $b$  contains  $cmd_M$  and no  $qc \in comchange$  refers to  $b$  **then**  
 49:        $comchange \leftarrow comchange \cup commitQC'$   
 50:     **for each** new block  $b' = \langle v', c, h'', batch \rangle$  led by  $b$  **do**  
 51:        $lc, lC \leftarrow c, \mathcal{C}$   
 52:        $c, \mathcal{C} \leftarrow \text{Deliver}(v', c, batch)$   
 53:       **if**  $c \neq lc$  **then**  
 54:          $b'' \leftarrow commitQC'$   
 55:          $gap \leftarrow \{b', \dots, b''\} / b'$   
 56:          $catch \leftarrow catch \cup gap$   
 57:         send  $\langle \text{ST}, v', \mathcal{H}(b'), prepareQC, lockedQC, commitQC \rangle$  to  $\mathcal{C}$   
 58:         send  $\langle \text{ST}, v', \mathcal{H}(b'), \perp, \perp, \perp \rangle$  to all replicas but  $\mathcal{C}$  and  $lC$   
 59:         **if**  $\mathcal{P}_i \in \mathcal{C}$  **then**  
 60:           wait until the state is up to date (Algorithm 4, lines 12-17)  
 61:         **break**  
 62:       **else**  
 63:         send  $\langle \text{ST}, v', \mathcal{H}(b'), \perp, \perp, \perp \rangle$  to all replicas but  $\mathcal{C}$   
 64:       send  $\langle \text{NEW-VRF-VIEW}, v \rangle$  to all replicas  
 65:       send  $\langle \text{NEW-VIEW}, v, c, prepareQC \rangle$  to  $\text{Leader}(v + 1)$   
 66:        $v \leftarrow v + 1$   
 67: **upon**  $\text{Timeout}(\Delta)$  **do**  $\triangleright$  VIEW-CHANGE  
 68:   send  $\langle \text{NEW-VRF-VIEW}, v \rangle$  to all replicas  
 69:   send  $\langle \text{NEW-VIEW}, v, c, prepareQC \rangle$  to  $\text{Leader}(v + 1)$   
 70:    $v \leftarrow v + 1$   
 71: **upon repeated**  $\text{Timeout}(\Delta)$  **do**  $\triangleright$  CATCH-UP

---

```

72:   denote the highest block by  $b' = \langle v', c', h', batch' \rangle$ 
73:   send  $\langle \text{CATCH-UP}, v', c' \rangle$  to  $\mathcal{C}$ 
74:   upon receiving  $\langle \text{CATCH-UP}, v', c' \rangle$  from  $\mathcal{P}_j$  do
75:     if  $\mathcal{P}_i \in \mathcal{C}_{c'}$  then
76:       denote the highest block related to  $\mathcal{C}_{c'}$  by  $b'' = \langle v'', c'', h'', batch'' \rangle$ 
77:       if  $v'' > v'$  then
78:          $hist \leftarrow$  all delivered blocks from view  $v' + 1$  to  $v''$ 
79:          $catch' \leftarrow$  all blocks in  $catch$  that are related to  $\mathcal{C}_{c'}$ 
80:         if  $c = c'$  then
81:            $qc \leftarrow \text{commitQC}$ 
82:         else
83:            $qc \leftarrow$  the  $\text{commitQC}$  in  $\text{comchange}$  that are related to  $\mathcal{C}_{c'}$ 
84:         send  $\langle \text{HIST}, v', c', hist, catch', qc \rangle$  to  $\mathcal{P}_j$ 
85:     upon receiving  $\langle \text{HIST}, v', c, hist, catch', qc \rangle$  from  $\mathcal{P}_j$  do
86:       denote the highest block by  $b'' = \langle v'', c, h'', batch'' \rangle$ 
87:       if the first block in  $hist$  directly extends  $b'' \wedge$  all blocks in  $hist$  and  $catch'$  are
         related to  $\mathcal{C}_{c'}$  and form a chain then
88:         parse  $qc$  as  $\langle \text{COMMIT}, v''', c, h, \mathcal{S}_c \rangle$ 
89:         denote the last block in  $hist \cup catch'$  as  $b$ 
90:         parse  $b$  as  $\langle v''', c, h', batch \rangle$ 
91:         if  $\mathcal{H}(b) = h \wedge \text{VrfySig}(\text{COMMIT} || v''' || c || h, \mathcal{S}_c) = 1$  then
92:           execute line 46-63
93:          $v \leftarrow v''' + 1$ 

```

---

#### B.4 The State Transfer Process

The state transfer process helps replicas to synchronize blocks and QCs from others, as shown in Algorithm 4. If  $\mathcal{P}_i$  is not in the committee, it waits for  $\lambda - t$  ST messages from the committee (line 1). Then  $\mathcal{P}_i$  obtains the corresponding block  $b$  from the BLOCK messages it receives in Algorithm 3, line 17 (line 2). If  $b$  directly extends the highest block  $\mathcal{P}_i$  records (line 3), then  $\mathcal{P}_i$  delivers the block (line 5). Moreover, if  $\mathcal{P}_i$  is in the next committee,  $\mathcal{P}_i$  updates its  $\text{prepareQC}$ ,  $\text{lockedQC}$ , and  $\text{commitQC}$  to the highest ones received and then starts to participate in the consensus (lines 6-10). Finally  $\mathcal{P}_i$  updates its view number (line 11).

For a committee member  $\mathcal{P}_i$ , it waits for ST messages only when the committee rotates, and it is in the next committee (Algorithm 3, lines 59-60). Then  $\mathcal{P}_i$  updates its  $\text{prepareQC}$ ,  $\text{lockedQC}$ , and  $\text{commitQC}$  to the highest it receives in ST messages and starts to participate in the consensus (lines 13-16). Finally,  $\mathcal{P}_i$  updates its view number (line 17).

#### B.5 Protocol Analysis

**Security proof.** Our Walnut-HS is secure under a weakly adaptive adversary. The detailed proof is shown in Appendix C.

**Communication complexity.** Our Walnut-HS involves one-to-all,  $\kappa$ -to- $\kappa$ , and  $\kappa$ -to-all communication, thus the message complexity is  $O(\kappa n)$ . For communication complexity, we measure it in terms of bit length to better illustrate the communication cost of our protocol.

**Algorithm 4** State Transfer

---

**Init:**  $\mathcal{P}_i$  initializes the current state transfer view number  $v_{st}$  as 0. The committee number  $c$  and committee  $\mathcal{C}$  are the same as that in Algorithm 3.

---

**As a replica  $\mathcal{P}_i$  not in  $\mathcal{C}$ :**

- 1: **upon** receiving  $\lambda - t \langle \text{ST}, v', h, *, *, * \rangle$  from  $\mathcal{C}$  s.t.  $v' \geq v_{st}$  **do**
- 2:   get the corresponding  $\langle \text{BLOCK}, v', c, b \rangle$  received s.t.  $h = \mathcal{H}(b)$
- 3:   **if**  $b$  directly extends the highest block  $\mathcal{P}_i$  records **then**
- 4:     parse  $b$  as  $\langle v', c, h', batch \rangle$
- 5:      $c, \mathcal{C} \leftarrow \text{Deliver}(v', c, batch)$
- 6:     **if**  $\mathcal{P}_i \in \mathcal{C}$  **then**
- 7:        $prepareQC \leftarrow$  the highest  $prepareQC'$  received
- 8:        $lockedQC \leftarrow$  the highest  $lockedQC'$  received
- 9:        $commitQC \leftarrow$  the highest  $commitQC'$  received
- 10:      Start to participate in the consensus
- 11:       $v_{st} \leftarrow v' + 1$

**As a replica  $\mathcal{P}_i$  in  $\mathcal{C}$  and the next committee:**

- 12: **upon** receiving  $\lambda - t \langle \text{ST}, v', h, *, *, * \rangle$  from  $\mathcal{C}$  s.t.  $h = \mathcal{H}(b) \wedge b$  is the highest block  $\mathcal{P}_i$  records  $\wedge b$  contains a  $cmd_M$  **do**
- 13:    $prepareQC \leftarrow$  the highest  $prepareQC'$  received
- 14:    $lockedQC \leftarrow$  the highest  $lockedQC'$  received
- 15:    $commitQC \leftarrow$  the highest  $commitQC'$  received
- 16:   Start to participate in the consensus
- 17:    $v_{st} \leftarrow v' + 1$

---

For the committee election process, each replica holding a valid random number sends a message  $\langle \text{VRF}, v_{ce}, c, \rho, \pi \rangle$  to the current committee, which is a  $\kappa$ -to- $\kappa$  communication on average. The length of the VRF evaluation is  $O(\kappa)$ . Thus the communication complexity is  $O(\kappa^3)$ .

For the M- $\Pi_{\text{HS}}$  process, in the PREPARE phase, the leader sends a block to all replicas and sends  $highQC$  to the committee members. The size of a  $highQC$  is  $\kappa^2$ . Thus the communication is  $O(nL + \kappa^3)$ , where  $L$  is the size of a block. In the PRE-COMMIT and COMMIT phase, the leader collects votes from the committee and sends the generated  $QC$  to the committee. The size of a  $QC$  is  $\kappa^2$ . Thus the communication is  $O(\kappa^3)$ . In the DECIDE phase, all committee members send their  $prepareQC$  to the next leader at last. Thus the communication is still  $O(\kappa^3)$ . The complexity of ST messages is measured in the state transfer process.

For the state transfer process, the committee members send the hash value of the delivered block to all non-committee members. In the meanwhile, they send  $prepareQC$ ,  $lockedQC$ , and  $commitQC$  to the next committee. Thus the communication is  $O(\kappa^2 n + \kappa^4)$ , where the  $O(\kappa^4)$  only occurs when committee rotation happens.

Therefore, the overall communication complexity of our Walnut-HS is  $O(nL + \kappa^2 n + \kappa^4)$ . In fact, the frequency of committee rotation can be adjusted as needed. For lower communication cost, we can simply set the frequency s.t. committee



rotation occurs once every  $\kappa$  views on average. Thus the communication complexity becomes  $O(nL + \kappa^2 n)$ .

## Appendix C Security Proof for Walnut-HS

We prove that our Walnut-HS satisfies the five properties we define in Section 3.6. We first provide a sketch of the proof and then show the detailed proof.

**Proof sketch.** We briefly show how Walnut-HS satisfies the five properties respectively.

*Random Election.* To prove random election, the crux is to show that each delivered block is verified by at least  $t + 1$  correct replicas (lemma 5), thus the  $cmd_M$  contained in the block must be valid. We first prove that a correct replica receiving a *commitQC* implies that at least  $t + 1$  correct replicas have received a corresponding *lockedQC*, thus a corresponding *prepareQC* (lemma 4 and collary 1). Then we show that the validity of a delivered block is guaranteed, thus ensuring the randomness of the selected replicas by the randomness of VRF (theorem 3).

*Same-Com Safety.* The same-com safety is guaranteed by the safety of HotStuff, as shown in theorem 4.

*Same-Com Liveness.* The same-com liveness is also ensured by the liveness of HotStuff, as shown in theorem 5.

*Smooth Rotation.* We divide smooth rotation into three parts and prove each one respectively.

Lemma 9 proves that if a correct committee member delivers a block  $b$  in view  $v$ , then all correct committee members will eventually deliver the block. The crux is to show that: (1) Any block  $b'$  related to the same committee and is higher than  $b$  must extend  $b$  (lemma 7); (2) Only  $b$  can be delivered in view  $v$  (lemma 8). Thus, all correct committee members will finally deliver  $b$  either by directly receiving a *commitQC* or synchronizing with others when experiencing repeated timeout.

Lemma 10 then proves that if  $b$  contains  $cmd_M$ , then  $b$  is the last block delivered by  $C_c$ . This is mainly due to the "refer to" relation we define. More specifically, if  $\mathcal{P}_i$  receives a *commitQC* ( $b \stackrel{*}{\leftarrow} commitQC$ ), then committee rotation is triggered after the delivery of  $b$ . If  $b \leftarrow commitQC$ , the proof is completed. Otherwise, since the delivery of subsequent blocks is interrupted (Algorithm 3, line 61),  $b$  is still the last block delivered by  $C_c$ , thus the lemma still holds.

Finally, lemma 12 proves that the next committee eventually delivers new blocks, and the first block must directly extend  $b$ . This is mainly due to the state transfer across committees. More specifically, when committee rotates, members in the current committee will send their *prepareQC*, *lockedQC*, and *commitQC* to the next committee (Algorithm 3, line 57). Thus all replicas in the next committee can update their states to the latest and then start to propose and deliver new blocks. Actually,  $b$  can be viewed as a genesis block to the new committee, thus the lemma holds.

*State Synchronization.* The proof for state synchronization is quite natural. First, correct committee members will send the hash value of the delivered block  $b$  to non-committee replicas (Algorithm 3, lines 57, 58, 63), thus each non-committee replica will eventually receive enough messages and deliver  $b$  (Algorithm 4, line 1). Second, a correct non-committee replica will only deliver  $b$  when receiving enough ST messages (Algorithm 4, line 1). Thus theorem 7 holds. **The detailed proof.** Next we give the detailed proof.

**Lemma 4** *If  $t_c < \lambda/3$  holds for the committee  $\mathcal{C}_c$ , the following holds: (1) if a correct replica in  $\mathcal{C}_c$  receives a *commitQC* referring to  $b$  related to  $\mathcal{C}_c$ , at least  $t+1$  correct replicas in  $\mathcal{C}_c$  have received a *lockedQC* referring to  $b$  before sending a *V-COM* message. (2) if a correct replica in  $\mathcal{C}_c$  receives a *lockedQC* referring to  $b$  related to  $\mathcal{C}_c$ , at least  $t+1$  correct replicas in  $\mathcal{C}_c$  have received a *prepareQC* referring to  $b$  before sending a *V-PRE-COM* message.*

*Proof.* Assume a correct replica  $\mathcal{P}_i$  receives a *commitQC* referring to  $b$ . Since a *commitQC* contains  $\lambda-t$  votes and  $t_c \leq t < \lambda/3$  holds for  $\mathcal{C}_c$ , at least  $t+1$  correct replicas in  $\mathcal{C}_c$  have sent a *V-COM* message. Before sending such a message, they must have received a *lockedQC* referring to  $b$  (Algorithm 3, lines 37–42).

Assume a correct replica  $\mathcal{P}_j$  receives a *lockedQC* referring to  $b$ . Similar to the above analysis, we know that at least  $t+1$  correct replicas in  $\mathcal{C}_c$  have received a *prepareQC* referring to  $b$  before sending a *V-PRE-COM* message (Algorithm 3, lines 31–36).

**Collary 1** *If  $t_c < \lambda/3$  holds for the committee  $\mathcal{C}_c$ , and a correct replica in  $\mathcal{C}_c$  receives a *commitQC* referring to  $b$  related to  $\mathcal{C}_c$ , at least  $t+1$  correct replicas in  $\mathcal{C}_c$  have received a *prepareQC* referring to  $b$  before sending a *V-PRE-COM* message.*

*Proof.* Assume a correct replica  $\mathcal{P}_i$  receives a *commitQC* referring to  $b$ . By Lemma 4, at least  $t+1$  correct replicas in  $\mathcal{C}_c$  have received a *lockedQC* referring to  $b$ . Applying Lemma 4 again, we know that at least  $t+1$  correct replicas have received a *prepareQC* referring to  $b$  before sending a *V-PRE-COM* message.

**Lemma 5** *If  $t_c < \lambda/3$  holds for the committee  $\mathcal{C}_c$  and a correct replica in  $\mathcal{C}_c$  delivers a block  $b$ , then at least  $t+1$  correct replicas in  $\mathcal{C}_c$  have received a *prepareQC* referring to  $b$  and  $b$  is verified by at least  $t+1$  correct replicas in  $\mathcal{C}_c$ .*

*Proof.* Assume a correct replica  $\mathcal{P}_i$  in  $\mathcal{C}_c$  delivers  $b$ . There are three cases: (1)  $\mathcal{P}_i$  has received a *commitQC* referring to  $b$  and delivers  $b$  directly (Algorithm 3, lines 43–63); (2)  $\mathcal{P}_i$  delivers  $b$  through the delivery of a block  $b'$  which is related to  $\mathcal{C}_c$  and extends  $b$  (Algorithm 3, lines 50–63); (3)  $\mathcal{P}_i$  experienced a repeated timeout and finally catch up with other replicas (Algorithm 3, lines 71–93).

Case (1): If  $\mathcal{P}_i$  has received a *commitQC* referring to  $b$ , by Collary 1, at least  $t+1$  correct replicas in  $\mathcal{C}_c$  have received a *prepareQC* referring to  $b$ . Since a *prepareQC* contains  $\lambda-t$  votes, at least  $t+1$  correct replicas in  $\mathcal{C}_c$  have verified  $b$  and the corresponding *highQC* before voting for it (Algorithm 3, lines 27–30).

Case (2): If  $\mathcal{P}_i$  has received a *commitQC* referring to  $b'$ , similar to the analysis for case (1), we know that at least  $t+1$  correct replicas in  $\mathcal{C}_c$  have verified  $b'$  and the corresponding *highQC* before voting for it. If  $b$  is the parent of  $b'$ , then this *highQC* is a *prepareQC* referring to  $b$ , thus at least  $t+1$  correct replicas in  $\mathcal{C}_c$  have verified  $b$ . If not, let all the blocks from  $b$  to  $b'$  be  $[b, b_1, \dots, b_m, b']$ . From the above analysis, we know that this *highQC* is a *prepareQC* referring to  $b_m$ . Thus at least  $t+1$  correct replicas have verified  $b_m$  and the *highQC* referring to the parent of  $b_m$ . Repeat this logic, and eventually, we know that at least  $t+1$  correct replicas in  $\mathcal{C}_c$  have received a *prepareQC* referring to  $b$  and  $b$  is verified by at least  $t+1$  correct replicas in  $\mathcal{C}_c$ .

Case (3): When catching up with other replicas,  $\mathcal{P}_i$  sends a CATCH-UP message containing the view of its highest block and the committee number to  $\mathcal{C}_c$  (Algorithm 3, lines 71–73). Correct replicas in  $\mathcal{C}_c$  will reply to  $\mathcal{P}_i$  with all blocks related to  $\mathcal{C}_c$  they have delivered together with the highest *commitQC* related to  $\mathcal{C}_c$  (Algorithm 3, lines 74–84). Thus  $\mathcal{P}_i$  will deliver new blocks by the *commitQC* related to  $\mathcal{C}_c$  it receives (Algorithm 3, lines 85–93). If the *commitQC* refers to  $b$ , then the proof is similar to that shown in Case (1). If not, then  $b$  is delivered through the delivery of a block  $b'$ , which is related to  $\mathcal{C}_c$  and extends  $b$ . Thus the proof is similar to that shown in Case (2).

**Theorem 3** (Random Election). *If  $t_c < \lambda/3$  holds for the current committee  $\mathcal{C}_c$  and a correct replica in  $\mathcal{C}_c$  delivers a block  $b$  containing a  $\text{cmd}_M$ , then the corresponding  $\lambda$  replicas referred to by  $\text{cmd}_M$  can be viewed as randomly selected from the system.*

*Proof.* For committee election, when a correct replica in  $\mathcal{C}_c$  has a view change, it sends a NEW-VRF-VIEW message to all replicas (Algorithm 3, lines 64, 68). Thus every correct replica eventually receives  $\lambda - t$  NEW-VRF-VIEW messages in a view  $v'$  (Algorithm 2, line 1) and updates their view for committee election (Algorithm 2, line 2). Thus the view for committee election can be viewed as synchronized with the view for consensus. In each view, every replica  $\mathcal{P}_i$  calls  $\text{VRFGen}()$  to generate a random number and check whether it is a candidate for the next committee (Algorithm 2, line 3). If so,  $\mathcal{P}_i$  sends  $\langle \text{VRF}, \rho, \pi \rangle$  to the current committee  $\mathcal{C}$  (Algorithm 2, lines 4–5). The adversary cannot know what exactly the random number  $\rho$  included in the VRF message is before it receives the message. Thus it cannot know who the next committee members are until it receives the VRF messages. Since we assume "atomic sends", it is either too early or too late to decide which messages to delay. Thus the adversary cannot targetedly affect the VRF messages among honest replicas.

Since a correct replica in  $\mathcal{C}_c$  delivers  $b$  and  $t_c < \lambda/3$  holds for  $\mathcal{C}_c$ , by Lemma 5,  $b$  is verified by at least  $t+1$  correct replicas in  $\mathcal{C}_c$ . This implies that the  $\lambda$  VRF messages contained in  $b$  are verified by the  $t+1$  correct replicas to be the smallest among all the VRF messages received in that view (Algorithm 1, lines 9–12). Since the adversary cannot targetedly affect the VRF messages among honest replicas, these random numbers contained in the  $\lambda$  VRF messages can be viewed as the smallest among all the VRF messages in that view. Since  $\rho$  is randomly generated,

the corresponding  $\lambda$  replicas can be viewed as randomly selected from the whole system.

**Theorem 4** (Same-Com Safety). *If  $t_c < \lambda/3$  holds for the current committee  $\mathcal{C}_c$  and a correct replica in  $\mathcal{C}_c$  delivers a command  $\text{cmd}$  before delivering  $\text{cmd}'$ , then no correct replica in  $\mathcal{C}_c$  delivers a command  $\text{cmd}'$  without first delivering  $\text{cmd}$ .*

*Proof.* When considering a single committee, the consensus process is very similar to HotStuff. Since we do not change the prepare, pre-commit, and commit logic of HotStuff, this theorem naturally holds by the safety of HoStuff.

**Theorem 5** (Same-Com Liveness). *If  $t_c < \lambda/3$  holds for the current committee  $\mathcal{C}_c$  and the committee does not change, if a command  $\text{cmd}$  is submitted to all correct replicas in  $\mathcal{C}_c$ , then all correct replicas in  $\mathcal{C}_c$  eventually deliver  $\text{cmd}$ .*

*Proof.* When considering the latest committee which does not change, the consensus process is very similar to HotStuff. Thus the theorem holds by the liveness of HotStuff.

**Lemma 6** *If  $t_c < \lambda/3$  holds for the committee  $\mathcal{C}_c$ , a correct replica  $\mathcal{P}_i$  in  $\mathcal{C}_c$  delivers a block  $b$  in view  $v$ , another correct replica  $\mathcal{P}_j$  in  $\mathcal{C}_c$  receives a  $\text{prepareQC}$   $qc'$  in view  $v'$  ( $v' > v$ ), then  $qc'$  extends  $b$ .*

*Proof.*  $\mathcal{P}_i$  and  $\mathcal{P}_j$  are both in  $\mathcal{C}_c$ , thus we only consider  $\mathcal{C}_c$ . Similar to the analysis in the proof of Lemma 5, there are three cases for the delivery of  $b$ . However, in all cases,  $\mathcal{P}_i$  has received a  $\text{commitQC}$   $qc$  referring to  $b$  or  $b_x$  which extends  $b$ . W.l.o.g., we assume  $b = b_x$ . Let the set of replicas that send a V-COM message for  $b$  in view  $v$  be  $S_1$ .

First, we show that when  $qc$  forms, it is the highest quorum certificate. Before  $qc$  forms, there exists no quorum certificates  $qc''$  higher than view  $v$ . Otherwise, let the set of replicas that contribute to  $qc''$  be  $S_2$ . All correct replicas in  $S_2$  are already in a view higher than  $v$ .  $S_1$  and  $S_2$  intersects in at least one correct replica in  $\mathcal{C}_c$ . Thus, at least one correct replica in a view higher than  $v$  sends a message with view number  $v$ , a contradiction. Similarly, there exist no quorum certificates referring to a conflicting block  $b''$  in view  $v$ . Otherwise, at least one correct replica in  $\mathcal{C}_c$  would have voted twice in view  $v$ , a contradiction.

Second, we show that after  $qc$  forms, only a  $\text{prepareQC}$ , which extends  $b$ , can form. Towards a contradiction, assume there exists a  $\text{prepareQC}$  which refers to a block that conflicts with  $b$ . Let the lowest such  $\text{prepareQC}$  be  $qc'''$  referring to  $b'''$  and the set of replicas which contribute to  $qc'''$  be  $S_3$ . By Lemma 4, since  $\mathcal{P}_i$  receives  $qc$ , at least  $t + 1$  correct replicas have received a  $\text{lockedQC}$  referring to  $b$  and lock on  $b$ . Here, these correct replicas are considered to be in  $S_1$ . Since all quorum certificates lower than  $qc'''$  and higher than  $qc$  refer to a block extending  $b$ , all the correct replicas in  $S_1$  still lock on  $b$  or a block extending  $b$ . Moreover, since  $b'''$  conflicts with  $b$  and  $qc'''$  is the lowest quorum certificate conflicting with  $b$ ,  $b'''$  together with the  $\text{highQC}$  referring to its parent satisfy neither the safety rule (Algorithm 1, line 18) nor the liveness rule (Algorithm 1, line 19).  $S_1$  and

$S_3$  intersect in at least one correct replica. Thus at least one correct replica has locked on  $b$  or its extension but still votes for  $b'''$ . A contradiction.

**Collary 2** *If  $t_c < \lambda/3$  holds for the committee  $\mathcal{C}_c$  and a correct replica  $\mathcal{P}_i$  in  $\mathcal{C}_c$  delivers a block  $b$  in view  $v$ . The following holds: (1) If another correct replica  $\mathcal{P}_j$  in  $\mathcal{C}_c$  receives a lockedQC  $qc'$  in view  $v'$  ( $v' > v$ ), then  $qc'$  extends  $b$ ; (2) If another correct replica  $\mathcal{P}_k$  in  $\mathcal{C}_c$  receives a commitQC  $qc''$  in view  $v''$  ( $v'' > v$ ), then  $qc''$  extends  $b$ ;*

*Proof.* We prove this collary by contradiction. Assume  $qc'$  and  $qc''$  both conflict with  $b$ , then the blocks they refer to ( $b'$  and  $b''$  respectively) both conflict with  $b$ .

Since  $\mathcal{P}_j$  receives  $qc'$ , by Lemma 4, at least  $t+1$  correct replicas in  $\mathcal{C}_c$  have received a *prepareQC*  $qc'_p$  referring to  $b'$  in view  $v'$ . Since  $b'$  conflicts with  $b$ ,  $qc'_p$  conflicts with  $b$ . However, by Lemma 6,  $qc'_p$  must extend  $b$ , thus a contradiction.

Since  $\mathcal{P}_k$  receives  $qc''$ , by Collary 1, at least  $t+1$  correct replicas in  $\mathcal{C}_c$  have received a *prepareQC*  $qc''_p$  referring to  $b''$  in view  $v''$ . Since  $b''$  conflicts with  $b$ ,  $qc''_p$  conflicts with  $b$ . However, by Lemma 6,  $qc''_p$  must extend  $b$ , thus a contradiction.

**Lemma 7** *If  $t_c < \lambda/3$  holds for the committee  $\mathcal{C}_c$ , a correct replica  $\mathcal{P}_i$  in  $\mathcal{C}_c$  delivers a block  $b$  in view  $v$ , another correct replica  $\mathcal{P}_j$  in  $\mathcal{C}_c$  delivers a block  $b'$  in view  $v'$  ( $v' > v$ ), then  $b'$  extends  $b$ .*

*Proof.*  $\mathcal{P}_i$  and  $\mathcal{P}_j$  are both in  $\mathcal{C}_c$ , thus we only consider the  $c$ -th committee. Similar to the analysis in the proof of Lemma 5, there are three cases for the delivery of  $b'$ . However, in all the three cases,  $\mathcal{P}_j$  has received a *commitQC*  $qc'$  referring to  $b'$  or  $b_y$  which extends  $b'$ . W.l.o.g., we assume  $b' = b_y$ . By Collary 2,  $qc'$  must extend  $b$ . Thus,  $b'$  extends  $b$ .

**Lemma 8** *If  $t_c < \lambda/3$  holds for the committee  $\mathcal{C}_c$  and a correct replica  $\mathcal{P}_i$  in  $\mathcal{C}_c$  delivers a block  $b$  in view  $v$ , then  $b$  is the only block that can be delivered in view  $v$ .*

*Proof.* Towards a contradiction, we assume a correct replica  $\mathcal{P}_j$  in  $\mathcal{C}_c$  delivers  $b'$  in view  $v$  and  $b'$  conflicts with  $b$ . Similar to the analysis in the proof of Lemma 5, there are three cases for the delivery of  $b$  and  $b'$ , respectively. However, if a correct replica delivers a block by catching up with others, there must be at least one correct replica that has delivered the block by receiving a *commitQC* directly. Thus, we can omit the third case.

Now, there are two cases for  $\mathcal{P}_i$ : (a)  $\mathcal{P}_i$  has received a *commitQC*  $qc$  referring to  $b$ ; (b)  $\mathcal{P}_i$  delivers  $b$  through the delivery of a block  $b_x$  which is related to  $\mathcal{C}_c$  and extends  $b$ . Similarly, there are two cases for  $\mathcal{P}_j$ : (c)  $\mathcal{P}_j$  has received a *commitQC*  $qc'$  referring to  $b'$ ; (d)  $\mathcal{P}_j$  delivers  $b'$  through the delivery of a block  $b_y$  which is related to  $\mathcal{C}_c$  and extends  $b'$ .

Case (1): Case (a) for  $\mathcal{P}_i$  and Case (c) for  $\mathcal{P}_j$ . Let the set of replicas that vote for  $b$  be  $S_1$ , and the set of replicas that vote for  $b'$  be  $S_2$ . Since  $qc$  and  $qc'$  both

contain  $\lambda - t$  votes,  $S_1$  and  $S_2$  must intersect in at least one correct replica  $\mathcal{P}_k$ . Thus,  $\mathcal{P}_k$  has voted for both  $b$  and  $b'$  in view  $v$ , a contradiction.

Case (2): Case (a) for  $\mathcal{P}_i$  and Case (d) for  $\mathcal{P}_j$ . Let  $b_y$  be delivered in view  $v'$ , then  $v' > v$ . By our assumption,  $b_y$  contradicts with  $b$  since  $b'$  contradicts with  $b$ . However, by Lemma 7,  $b_y$  must be an extension of  $b$ , a contradiction.

Case (3): Case (b) for  $\mathcal{P}_i$  and Case (c) for  $\mathcal{P}_j$ . This case is similar to Case (2) and we omit the details here.

Case (4): Case (b) for  $\mathcal{P}_i$  and Case (d) for  $\mathcal{P}_j$ . Let  $b_x$  be delivered in view  $v_x$  and  $b_y$  in view  $v_y$ . If  $v_x = v_y$ , by the analysis in Case (1),  $b_x = b_y$  and thus  $b = b'$ . If not, w.l.o.g., assume  $v_x < v_y$ . By our assumption,  $b'$  conflicts with  $b$ . However, by Lemma 7,  $b_y$  must extend  $b_x$ , a contradiction. Thus  $b = b'$ .

**Lemma 9** *If  $t_c < \lambda/3$  holds for the committee  $\mathcal{C}_c$  and a correct replica  $\mathcal{P}_i$  in  $\mathcal{C}_c$  delivers a block  $b$ , then all correct replicas in  $\mathcal{C}_c$  eventually deliver  $b$ .*

*Proof.* Similar to the analysis in the proof of Lemma 5, there are three cases for the delivery of  $b$ : (1)  $\mathcal{P}_i$  has received a *commitQC'* referring to  $b$ ; (2)  $\mathcal{P}_i$  delivers  $b$  through the delivery of a block  $b'$  which is related to  $\mathcal{C}_c$  and extends  $b$ ; (3)  $\mathcal{P}_i$  experienced a repeated timeout and finally catch up with a correct replica in  $\mathcal{C}_c$ . W.l.o.g., we only consider the former two cases since the first correct replica delivering  $b$  cannot deliver  $b$  by catching up with others. In the first two cases above,  $\mathcal{P}_i$  has received a *commitQC'* referring to  $b$  or a block  $b'$  which is related to  $\mathcal{C}_c$  and extends  $b$ . Denote this block by  $b_i$  and let  $b_i$  be delivered in view  $v_i$ .  $\mathcal{P}_i$  then updates its *commitQC* together with the set *catch* (Algorithm 3, lines 46, 56).

Now we consider another correct replica  $\mathcal{P}_j$  in  $\mathcal{C}_c$ . Since the committee number of  $\mathcal{P}_j$  is  $c$ ,  $\mathcal{P}_j$  can only directly deliver a block related to  $\mathcal{C}_c$ . There are three cases for the delivery of blocks higher than view  $v_i - 1$ : (1)  $\mathcal{P}_j$  delivers a block in view  $v_i$ ; (2)  $\mathcal{P}_j$  delivers a block in view  $v''$  ( $v'' > v_i$ ); (3)  $\mathcal{P}_j$  delivers no block higher than view  $v_i - 1$  and experiences repeated timeout.

Case (1): If  $\mathcal{P}_j$  delivers a block in view  $v_i$ , by Lemma 8, this block must be  $b_i$  and thus  $\mathcal{P}_j$  must have delivered  $b$ .

Case (2): If  $\mathcal{P}_j$  delivers a block  $b''$  in view  $v''$  ( $v'' > v_i$ ), by Lemma 7,  $b''$  must extend  $b$  and  $\mathcal{P}_j$  will deliver  $b$  through the delivery of  $b''$  (Algorithm 3, lines 50–63).

Case (3): If  $\mathcal{P}_j$  delivers no block higher than view  $v_i - 1$  and experiences repeated timeout,  $\mathcal{P}_j$  will send a CATCH-UP message to replicas in  $\mathcal{C}_c$  (Algorithm 3, lines 71–73). When a correct replica  $\mathcal{P}_k$  receives a CATCH-UP message from  $\mathcal{P}_j$ , it sends a HIST message which contains the blocks together with the proof (the highest *commitQC* related to  $\mathcal{C}_c$  it has) that  $\mathcal{P}_j$  falls behind to it (Algorithm 3, lines 74–84). Such a  $\mathcal{P}_k$  must exist; for example, it could be  $\mathcal{P}_i$ . If the highest block of  $\mathcal{P}_k$  is higher than  $b$ , by Lemma 7, it extends  $b$  and thus  $\mathcal{P}_j$  will deliver  $b$ . If the block is in the same height as  $b$ , by Lemma 8, it could only be  $b$  and  $\mathcal{P}_j$  still delivers  $b$ . If the height is lower than  $b$ , we know that  $\mathcal{P}_j$  will eventually deliver  $b$  by catching up with other correct replicas in  $\mathcal{C}_c$  or by repeating the catching up process.

**Lemma 10** *If  $t_c < \lambda/3$  holds for the committee  $\mathcal{C}_c$  and a correct replica  $\mathcal{P}_i$  in  $\mathcal{C}_c$  delivers a block  $b$  containing a  $cmd_M$ , then  $b$  is the last block delivered by the  $c$ -th committee.*

*Proof.* Consider a correct replica  $\mathcal{P}_j$  in  $\mathcal{C}_c$ . By Lemma 9, We know that  $\mathcal{P}_j$  will deliver  $b$ . Similar to the analysis in Lemma 5, there are three cases for the delivery of  $b$ .

Case (1): If  $\mathcal{P}_j$  has received a *commitQC* referring to  $b$ , it will deliver  $b$  (Algorithm 3, line 52). Since  $b$  contains a  $cmd_M$ , upon the delivery of  $b$ , the committee will be updated (Algorithm 1, lines 35–40). If  $\mathcal{P}_j$  is in  $\mathcal{C}_{c+1}$ , it continues to participate in the consensus process. However, the committee number of the new blocks and quorum certificates is restricted to be  $c+1$ , thus,  $\mathcal{P}_j$  can only deliver new blocks related to  $\mathcal{C}_{c+1}$ . Moreover, when delivering ancestors of a new block, the committee number is still restricted to be  $c+1$  (Algorithm 3, line 50). If  $\mathcal{P}_j$  is not in  $\mathcal{C}_{c+1}$ , it will deliver new blocks by the state transfer process (Algorithm 4, lines 1–11). However, these new blocks are all related to  $\mathcal{C}_{c+1}$ . Thus  $b$  is the last block related to  $\mathcal{C}_c$  delivered by  $\mathcal{P}_j$ .

Case (2):  $\mathcal{P}_j$  delivers  $b$  through the delivery of a block  $b'$  which is related to  $\mathcal{C}_c$  and extends  $b$  (Algorithm 3, lines 50–63). Since  $b$  contains a  $cmd_M$ , upon the delivery of  $b$ , the committee will be updated (Algorithm 1, lines 35–40). Then the blocks between  $b$  and  $b'$  will be stored to ensure the liveness of the protocol (Algorithm 3, lines 55–56) but the delivery of these blocks will be interrupted (Algorithm 3, line 61). Then, similar to the analysis in Case (1),  $b$  is the last block related to  $\mathcal{C}_c$  delivered by  $\mathcal{P}_j$ .

Case (3):  $\mathcal{P}_j$  experiences a repeated timeout and finally catches up with a replica  $\mathcal{P}_{k_1}$  (Algorithm 3, lines 71–93).  $\mathcal{P}_j$  will deliver the blocks it falls behind in the same order as  $\mathcal{P}_{k_1}$ . Since the message from  $\mathcal{P}_{k_1}$  can be verified by  $\mathcal{P}_j$ , the blocks and the quorum certificate do not conflict with those of any correct replica. Thus  $\mathcal{P}_{k_1}$  can be viewed as a correct replica. If  $\mathcal{P}_{k_1}$  delivers  $b$  by Case (1) or (2), from the above analysis, we know that  $b$  is the last block related to  $\mathcal{C}_c$  delivered by  $\mathcal{P}_{k_1}$  and thus the last block related to  $\mathcal{C}_c$  delivered by  $\mathcal{P}_j$ . If  $\mathcal{P}_{k_1}$  delivers  $b$  by catching up with another replica, let it be  $\mathcal{P}_{k_2}$  and repeat the above logic. Finally, we can get a series of replicas  $[\mathcal{P}_{k_1}, \mathcal{P}_{k_2}, \dots, \mathcal{P}_{k_m}]$ , of which  $\mathcal{P}_{k_m}$  must deliver  $b$  by either Case (1) or Case (2). Thus  $\mathcal{P}_j, \mathcal{P}_{k_1}, \dots, \mathcal{P}_{k_{m-1}}$  will deliver blocks in the same order as  $\mathcal{P}_{k_m}$ , then  $b$  is the last block related to  $\mathcal{C}_c$  delivered by  $\mathcal{P}_j$ .

**Lemma 11** *If  $t_c < \lambda/3$  holds for the committee  $\mathcal{C}_c$  and a correct replica  $\mathcal{P}_i$  in  $\mathcal{C}_c$  delivers a block  $b$  containing a  $cmd_M$ , before the first *prepareQC*  $qc$  related to  $\mathcal{C}_{c+1}$  forms, the *prepareQC* and *lockedQC* of all correct replicas in  $\mathcal{C}_{c+1}$  refer to  $b$  after the delivery of  $b$ .*

*Proof.* Let  $\mathcal{P}_k$  be the first replica that delivers  $b$ .  $\mathcal{P}_k$  must deliver  $b$  by receiving a *commitQC*  $qc_c$  directly referring to either  $b$  or  $b_x$  which is related to  $\mathcal{C}_c$  and extends  $b$ . By our definition, since  $b$  contains a  $cmd_M$ , in either case,  $qc_c$  must refer to  $b$ . By Lemma 4, at least  $t+1$  correct replicas in  $\mathcal{C}_c$  have received a *lockedQC*

$qc_l$  referring to  $b$  and thus lock on  $b$  before sending a V-COM message. Let these correct replicas be  $S_1$ . By Lemma 6 and Collary 2, all quorum certificates related to  $\mathcal{C}_c$  and formed after  $qc_c$  must extend  $b$ . Moreover, since  $b$  contains a  $cmd_M$ , these quorum certificates all refer to  $b$ . Thus, before  $qc$  forms, all replicas in  $S_1$  continue to lock on  $b$ . Similarly, there exists a set of  $t+1$  correct replicas  $S_2$  such that their *prepareQC* all refer to  $b$  before  $qc$  forms.

When a correct replica in  $\mathcal{C}_c$  delivers  $b$ , since  $b$  contains a  $cmd_M$ , the committee changes (Algorithm 3, line 53). Thus it will send a ST message which contains the hash value of  $b$ , *prepareQC*, *lockedQC* and *commitQC* to  $\mathcal{C}_{c+1}$  (Algorithm 3, line 57). Let  $\mathcal{P}_l$  be a correct replica in  $\mathcal{C}_{c+1}$ . By Lemma 9, all correct replicas in  $\mathcal{C}_c$  will deliver  $b$  and send such a ST message. Thus  $\mathcal{P}_l$  will finally receive  $\lambda-t$  ST messages (Algorithm 4, line 1 or line 12). Let these  $\lambda-t$  replicas that send such a message be  $S_3$ . Since  $S_1$  contains  $t+1$  replicas,  $S_1$  and  $S_3$  intersect in at least one correct replica. In other words,  $\mathcal{P}_l$  has received at least one *lockedQC*  $qc'_l$  referring to  $b$  in a ST message. Thus the *lockedQC* of  $\mathcal{P}_l$  is updated by  $qc'_l$  or another higher *lockedQC*  $qc'_l$  (Algorithm 4, line 8 or line 14). Since  $qc'_l$  is formed after  $qc_c$ , by Collary 2,  $qc'_l$  still refers to  $b$ . Thus all correct replicas in  $\mathcal{C}_{c+1}$  will lock on  $b$  before  $qc$  forms. The analysis for the update of *prepareQC* is quite similar, and we omit the details. The result is that the *prepareQC* of all correct replicas in  $\mathcal{C}_{c+1}$  refer to  $b$  before  $qc$  forms.

**Lemma 12** *If  $t_c < \lambda/3$  holds for the committee  $\mathcal{C}_c$  and a correct replica  $\mathcal{P}_i$  in  $\mathcal{C}_c$  delivers a block  $b$  containing a  $cmd_M$ , if  $t_{c+1} < \lambda/3$  holds for the committee  $\mathcal{C}_{c+1}$ , a correct replica  $\mathcal{P}_j$  in  $\mathcal{C}_{c+1}$  eventually delivers some new valid blocks and the first block it delivers directly extends  $b$ .*

*Proof.* First, we prove that a correct replica  $\mathcal{P}_j$  in  $\mathcal{C}_{c+1}$  eventually delivers some new valid blocks. By Lemma 9 and Lemma 10, all correct replicas in  $\mathcal{C}_c$  will eventually deliver  $b$  and  $b$  is the last block delivered by  $\mathcal{C}_c$ . Thus, all correct replicas that are not in  $\mathcal{C}_c$  will receive  $\lambda-t$  ST messages, deliver  $b$  as the last block related to  $\mathcal{C}_c$  and update to  $\mathcal{C}_{c+1}$ . Thus all correct replicas in  $\mathcal{C}_{c+1}$  will finally enter the  $c+1$ -th committee. By Lemma 11, before the first *prepareQC*  $qc$  related to  $\mathcal{C}_{c+1}$  forms, the *prepareQC* and *lockedQC* of all correct replicas in  $\mathcal{C}_{c+1}$  refer to  $b$ .

Once the leader in a view is correct, it will propose a new block  $b'$  based on the highest *prepareQC*  $qc_h$  it receives (Algorithm 3, lines 5–17) and  $qc_h$  must refer to  $b$ . When a correct replica  $\mathcal{P}_k$  in  $\mathcal{C}_{c+1}$  receives a PREPARE message containing  $qc_h$  and  $b'$ , it calls *VrfyProp()* to check the proposal (Algorithm 3, line 28). The condition in Algorithm 1, line 20 is satisfied. Moreover, since  $\mathcal{P}_k$  locks on  $b$ , the safety rule (Algorithm 1, line 23) is also satisfied. Then  $\mathcal{P}_k$  and thus all correct replicas in  $\mathcal{C}_{c+1}$  will vote for  $b'$ , forming  $qc$ . Thus new quorum certificates referring to new blocks can form in the new committee.

After  $qc$  forms, let the highest *lockedQC* kept by all correct replicas be  $qc'_l$  referring to  $b''$ . If  $b'' = b$ , then no *lockedQC* forms in  $\mathcal{C}_{c+1}$ . Similar to the above analysis, correct replicas will continue to vote for new valid blocks and form new quorum certificates. Thus, eventually  $b'' \neq b$ .



If  $b'' \neq b$ , then  $qc'_l$  is formed in  $\mathcal{C}_{c+1}$ , by Lemma 4, at least  $t+1$  correct replicas have received a *prepareQC* referring to  $b''$ , thus their *prepareQC* refer to  $b''$  or a higher block. Let the set of such correct replicas be  $S_1$ . Replicas in  $S_1$  send their *prepareQC* to the leader in a NEW-VIEW message, and thus, the leader will receive a *prepareQC* referring to  $b''$  or a higher block from at least one of these replicas. Once the leader is correct, it will propose a new block based on the *highQC* it calculates, and the *highQC* must refer to  $b''$  or a higher block (Algorithm 3, lines 12–13). Since  $qc'_l$  is the highest *lockedQC*, thus the *lockedQC*  $qc'_l$  kept by a correct replica  $\mathcal{P}_l$  in  $\mathcal{C}_{c+1}$  is no higher than *highQC*. When  $\mathcal{P}_l$  receives a PREPARE message from the leader, it calls `VrfyProp()` to check the proposal (Algorithm 3, line 28). If  $qc'_l$  is in the same view as *highQC*, the safety rule is satisfied (Algorithm 1, line 18). If lower, the liveness rule must be satisfied (Algorithm 1, line 19). Then  $\mathcal{P}_l$  and thus all correct replicas in  $\mathcal{C}_{c+1}$  will vote for the new block. Eventually, in a view, a new block will be delivered.

Actually, when the committee changes, the last block related to  $\mathcal{C}_c$  can be viewed as a "genesis block" to the new committee  $\mathcal{C}_{c+1}$  and  $\mathcal{C}_{c+1}$  will start from the same state and continue the consensus based on the "genesis block"  $b$ .

Let the first block delivered by  $\mathcal{C}_{c+1}$  be  $b''$ , next we prove that  $b''$  directly extends  $b$ . By Lemma 5, at least  $t+1$  correct replicas in  $\mathcal{C}_{c+1}$  have received a *prepareQC* referring to  $b$  and  $b$  is verified by at least  $t+1$  correct replicas. Let the *prepareQC* along with  $b''$  in the PREPARE message be  $qc'_h$ . If  $qc'_h$  refers to a block  $b'''$  related to  $\mathcal{C}_{c+1}$ ,  $b'''$  will be delivered through the delivery of  $b''$  and thus  $b''$  is not the first block delivered by  $\mathcal{C}_{c+1}$ , a contradiction. Thus  $qc'_h$  must refer to  $b$ . Since  $b''$  and  $qc'_h$  are verified by at least  $t+1$  correct replicas, the parent of  $b''$  must be  $b$  (Algorithm 1, line 20). Thus  $b''$  must directly extend  $b$ .

**Theorem 6** (Smooth Rotation). *If  $t_c < \lambda/3$  holds for the committee  $\mathcal{C}_c$  and a correct replica  $\mathcal{P}_i$  in  $\mathcal{C}_c$  delivers a block  $b$ , then all correct replicas in  $\mathcal{C}_c$  eventually deliver  $b$ . If  $b$  contains a  $\text{cmd}_M$ , then  $b$  is the last block delivered by the  $c$ -th committee. Moreover, If  $t_{c+1} < \lambda/3$  holds for the committee  $\mathcal{C}_{c+1}$ , a correct replica  $\mathcal{P}_j$  in  $\mathcal{C}_{c+1}$  eventually delivers some new valid blocks and the first block it delivers directly extends  $b$ .*

*Proof.* Combining Lemma 9, Lemma 10 and Lemma 12, this theorem holds.

**Theorem 7** (State Synchronization). *(1) If  $t_c < \lambda/3$  holds for the committee  $\mathcal{C}_c$  and a correct replica in  $\mathcal{C}_c$  delivers a block  $b$  in view  $v$ , then all correct replicas in the system that are not in  $\mathcal{C}_c$  eventually deliver  $b$  in view  $v$ .*

*(2) If  $t_c < \lambda/3$  holds for the committee  $\mathcal{C}_c$  and a correct replica in the system delivers a block  $b$  related to the  $c$ -th committee in view  $v$ , then at least one correct replica in  $\mathcal{C}_c$  has delivered  $b$  in view  $v$ .*

*Proof.* (1) By Lemma 9, all correct replicas in  $\mathcal{C}_c$  will deliver  $b$  in view  $v$ . These replicas will send a ST message which contains the hash value of  $b$  and other necessary data to all replicas that are not in  $\mathcal{C}_c$  (Algorithm 3, lines 57, 58, 63). Thus, all correct replicas that are not in  $\mathcal{C}_c$  will finally receive  $\lambda-t$  ST messages (Algorithm 4, line 1) and deliver  $b$  in view  $v$  (Algorithm 4, line 5).

(2) Consider two cases for the replica  $\mathcal{P}_i$  that delivers  $b$  in view  $v$ . If  $\mathcal{P}_i$  is in  $\mathcal{C}_c$ , then the theorem naturally holds. If not,  $\mathcal{P}_i$  must deliver  $b$  by receiving  $\lambda - t$  ST messages from  $\mathcal{C}_c$  (Algorithm 4, line 1). Among these messages, at least  $t+1$  are from correct replicas in  $\mathcal{C}_c$ . Then at least  $t+1$  correct replicas in  $\mathcal{C}_c$  has delivered  $b$  in view  $v$ .

## Appendix D The Throughput of Walnut-HS and HotStuff

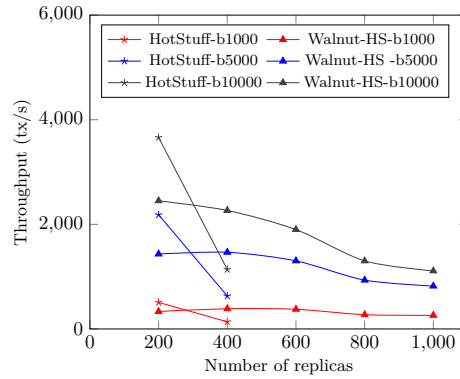


Fig. 6: Throughput for different batch sizes as  $n$  grows. The committee size for Walnut-HS is 200.